Ingeniería de Software

Clase Práctica 4

Patrones de diseño orientado a objetos

Ejercicios evaluativos 9 y 12

Equipo 7

Bales

Integrantes

Integrante	GitHub	Ejercicio / inciso
Heily Rodríguez Rodríguez	`@heilyrodriguez225`	Ej. 9 – incisos a, b
Mauricio Brindo Carbo	`@Mackandal04`	
Enrique González Moreira	`@QuiquiMatCom2004`	Ej. 9 – incisos c, d y restricción
Ernesto Alejandro Soler Choong	`@solerch5510`	Ej. 12 – incisos a, b, c, d y restricción

Repositorio

https://github.com/Team-Bales/Ej-Evaluativos-CP4

Índice

1. Ejercicio 9: Sistema de Notificaciones de Bolsa **3**
1.1. a) Patrón . . . 3
1.1.1. Patrón seleccionado: Observer (Publicador / Suscriptor) . . . 3
1.1.2. Justificación . . . 3
1.1.3. Propuesta de solución: clases y relaciones . . . 4
1.1.4. ¿Por qué no usar Mediator? . . . 5
1.1.5. ¿Por qué no usar polling (consulta periódica)? . . . 6
1.2. b) Implementación . . . 6
1.2.1. Cómo ejecutarlo . . . 6
1.2.2. Estructura del código . . . 7
1.2.3. Regla de disparo . . . 7
1.2.4. Datos de la simulación . . . 8
1.2.5. Resultados . . . 9
1.2.6. Pruebas unitarias . . . 9
1.2.7. Decisiones de esta implementación . . . 10
1.3. c) Análisis . . . 10
1.3.1. ¿Qué pasa si un cliente no puede ser notificado? . . . 11
1.3.2. ¿Cómo evitar bloquear el hilo principal? . . . 14
1.3.3. ¿Observer es síncrono o asíncrono? . . . 16
1.3.4. Cambios respecto al código del inciso b . . . 18
1.3.5. Pruebas unitarias . . . 19
1.4. d) Rendimiento . . . 19
1.4.1. Cómo se midió . . . 20
1.4.2. Resultados . . . 21
1.4.3. ¿Cómo cambia el orden de complejidad? . . . 22
1.4.4. El paralelismo reduce el tiempo de reloj, no el orden . . . 23
1.4.5. ¿Thread Pool o Reactive Streams? . . . 24
1.4.6. Propuesta final . . . 25
1.4.7. Pruebas unitarias . . . 26
1.4.8. Ficheros de este inciso . . . 26
1.5. Restricción . . . 26
1.5.1. Respuesta corta . . . 27
1.5.2. El outbox: escribir antes de enviar . . . 27
1.5.3. Los estados durables . . . 28
1.5.4. Idempotencia . . . 28
1.5.5. Reintentos con backoff y *dead letter* persistido . . . 28
1.5.6. Reinicio y recuperación . . . 29
1.5.7. Dónde vive la persistencia . . . 29
1.5.8. Límites del diseño . . . 30
1.5.9. Demostración . . . 30
1.5.10. Pruebas . . . 31
1.5.11. Ficheros nuevos y modificados . . . 31

2. Ejercicio 12: Sistema de Procesamiento de Transacciones Bancarias **32**
2.1. a) Patrón . . . 32
2.1.1. Patrón seleccionado: Visitor . . . 32
2.1.2. Justificación . . . 32
2.1.3. Propuesta de solución: clases y relaciones . . . 33
2.1.4. ¿Por qué no usar Strategy? . . . 37
2.1.5. ¿Por qué no usar polimorfismo simple? . . . 38

2.1.6. Desventajas de Visitor y cómo se acotan . . . 38
2.1.7. Patrones relacionados . . . 39
2.1.8. Usos conocidos . . . 39
2.2. b) Implementación . . . 39
2.2.1. Cómo ejecutarlo . . . 39
2.2.2. Estructura del código . . . 40
2.2.3. Cada operación se comporta distinto según la cuenta . . . 40
2.2.4. «Todo o nada» en una transacción . . . 42
2.2.5. Extensibilidad: una operación nueva sin tocar las cuentas . . . 43
2.2.6. Resultados de la simulación . . . 43
2.2.7. Pruebas . . . 44
2.3. c) Integración . . . 45
2.3.1. Composite: grupos de cuentas . . . 45
2.3.2. Iterator: las transacciones de una cuenta . . . 46
2.3.3. ¿Cómo afecta el Composite a la ejecución de operaciones sobre grupos? . . . 47
2.3.4. Resultados de la simulación . . . 48
2.3.5. Pruebas . . . 49
2.4. d) Comparación . . . 50
2.4.1. Las dos métricas . . . 50
2.4.2. Escenarios de crecimiento . . . 51
2.4.3. ¿Cuándo conviene cada uno? . . . 51
2.4.4. ¿Qué pasa si las operaciones crecen mucho más rápido que los tipos de cuenta? . . . 52
2.5. Restricción . . . 52
2.5.1. Respuesta corta . . . 53
2.5.2. Paralelismo sobre cuentas distintas . . . 53
2.5.3. El estado mutable de los visitantes . . . 53
2.5.4. Atomicidad y consistencia en la misma cuenta . . . 54
2.5.5. Límites y alternativas . . . 56
2.5.6. Demostración . . . 56

1. Ejercicio 9: Sistema de Notificaciones de Bolsa

1.1. a) Patrón

Identifica el patrón de diseño más adecuado, justifica tu selección basándote en las características del problema y explica la propuesta de solución considerando las clases que intervienen y las relaciones entre ellas. ¿Por qué no usar Mediator o polling?

1.1.1. Patrón seleccionado: Observer (Publicador / Suscriptor)

Observer define una dependencia uno-a-muchos entre objetos: cuando un objeto cambia de estado, todos sus dependientes son notificados y se actualizan automáticamente (Gamma et al., 1994, p. 326).

1.1.2. Justificación

El libro de la «banda de los cuatro» (*Gang of Four*, GoF), la referencia clásica de los patrones de diseño, recomienda Observer cuando un cambio en un objeto obliga a cambiar otros y no se sabe cuántos son, y cuando un objeto debe poder notificar a otros sin hacer suposiciones sobre quiénes son (Gamma et al., 1994, pp. 327–328). Ambas condiciones se cumplen en este problema:

Característica del problema	**Cómo la resuelve Observer**
Relación uno-a-muchos: una acción, miles de clientes interesados.	El *Sujeto* (la acción) mantiene una lista de *Observadores* y les avisa a todos cuando cambia su precio.
No se sabe de antemano cuántos dependientes hay. Los clientes eligen acciones y umbrales y pueden cambiarlos en cualquier momento.	`Suscribir()` y `Desuscribir()` se llaman en tiempo de ejecución, sin tocar el código de la acción.
Notificación inmediata.	El cambio de precio dispara `Notificar()` en ese mismo instante; el cliente no tiene que preguntar.
La acción no debe conocer a sus suscriptores (bajo acoplamiento).	La acción solo conoce la interfaz `IObservadorPrecio`; no sabe si detrás hay un email, un SMS o un push.
Extensibilidad (nuevos canales: WhatsApp, Telegram, webhook…).	Se añade una nueva implementación de `ICanalNotificacion` sin modificar `Accion` (principio Abierto/Cerrado).

Tabla 1: Características del problema y cómo las resuelve Observer.

En .NET el patrón Observer viene incluso en la biblioteca base: `IObservable<T>` / `IObserver<T>` y la palabra clave `event`. Aquí se implementa de forma explícita para que se vean los roles del patrón.

1.1.3. Propuesta de solución: clases y relaciones

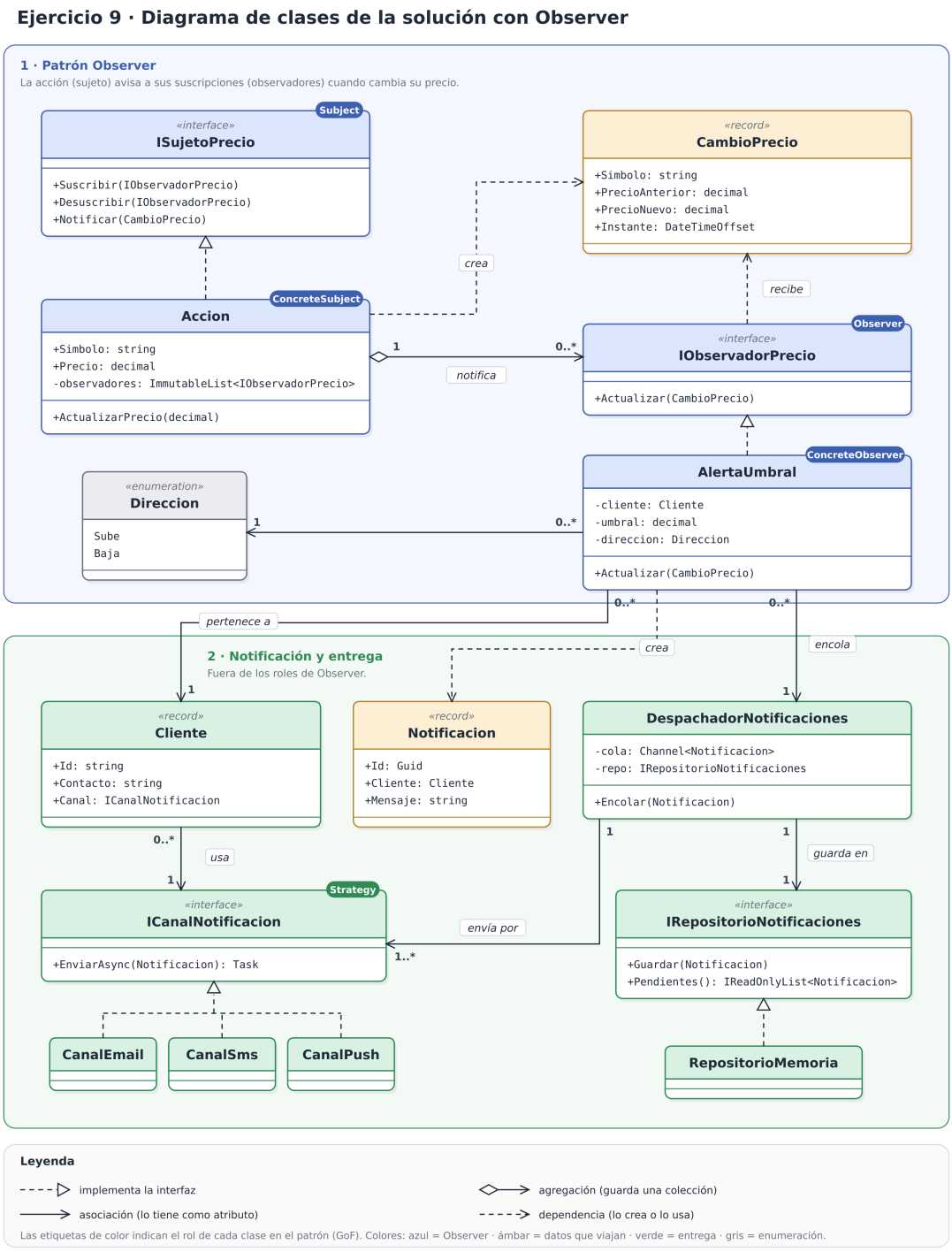


Figura 1: Diagrama de clases UML de la solución con Observer.

Roles del patrón.

- **Subject (`ISujetoPrecio`)**: interfaz con `Suscribir`, `Desuscribir` y `Notificar`.
- **ConcreteSubject (`Accion`)**: guarda el símbolo y el precio. Cuando el precio cambia, crea un `CambioPrecio` y notifica a sus observadores. Una acción tiene de 0 a muchos observadores.

- **Observer (`IObservadorPrecio`)**: interfaz con `Actualizar(CambioPrecio)`.
- **ConcreteObserver (`AlertaUmbral`)**: representa *una suscripción* de un cliente: acción, umbral y dirección (`Sube` o `Baja`). Decide si el cambio **cruza** su umbral y, si lo cruza, crea una `Notificacion`.

Clases de apoyo (fuera de los roles de Observer).

- **Canales (`CanalEmail`, `CanalSms`, `CanalPush`)**: formas de envío intercambiables detrás de `ICanalNotificacion`. Funciona como un **Strategy**, porque cada cliente elige su canal. Los canales **no son observadores**: el observador decide *si* hay que avisar, y el canal decide *cómo* se entrega el aviso.
- `DespachadorNotificaciones`: separa la detección del evento del envío real. Se encarga de que el envío sea asíncrono (inciso c), de los reintentos y de la persistencia (restricción). En este inciso solo se presenta el diseño; se implementa en los incisos y en la restricción que lo usan. Guarda las notificaciones en un `IRepositorioNotificaciones`; `RepositorioMemoria` es la implementación en memoria que usa el inciso b.

Decisiones de diseño.

- **El observador es la suscripción (`AlertaUmbral`), no el `Cliente`.** Un cliente puede tener muchos umbrales sobre muchas acciones. Así cada observador tiene una sola responsabilidad: evaluar *su* condición. El umbral vive dentro del observador, no en la acción, que no necesita saber qué le interesa a cada cliente.
- **Se notifica al cruzar el umbral.** Solo hay aviso cuando el precio **cruza** el umbral, no en cada *tick* en que siga por encima. Así el cliente no recibe el mismo aviso una y otra vez.

Modelo push o pull. GoF distingue dos extremos. En el modelo *push*, el sujeto envía a los observadores información detallada del cambio, la necesiten o no. En el modelo *pull*, el sujeto solo avisa que algo cambió, y cada observador pide después lo que necesita (Gamma et al., 1994, p. 332).

Esta solución usa el **modelo push**: `Notificar` envía un `CambioPrecio` con el símbolo, el precio anterior y el nuevo. Hay dos razones:

1. **Todos los observadores necesitan los mismos datos** (el precio anterior y el nuevo, para detectar el cruce). La desventaja que GoF señala para push, que el sujeto asume qué necesitan sus observadores, aquí no aplica.
2. **Con pull, cada uno de los miles de observadores volvería a consultar a la acción** después de cada aviso. Eso es justo la ineficiencia que GoF atribuye a ese modelo (Gamma et al., 1994, p. 332), y además el precio podría haber cambiado otra vez entre el aviso y la consulta.

1.1.4. ¿Por qué no usar Mediator?

- **Mediator resuelve otro problema.** Sirve para encapsular cómo interactúa un conjunto de objetos, y conviene cuando esos objetos se comunican "de forma bien definida pero compleja", con interdependencias desestructuradas (Gamma et al., 1994, pp. 305, 308). Un ejemplo es la comunicación muchos-a-muchos entre aviones del ejercicio 6. Aquí el flujo es **uno-a-muchos y en un solo sentido**: la acción emite y los clientes solo reciben. Los clientes nunca se comunican entre sí, así que no hay interacciones complejas que encapsular.

- **Un mediador central concentraría toda la lógica** en una sola clase: qué cliente quiere qué acción, con qué umbral y por qué canal. Habría que modificarla cada vez que se añade un tipo de alerta, lo que viola los principios Abierto/Cerrado y de Responsabilidad Única. Observer, en cambio, reparte esa lógica: cada `AlertaUmbral` sabe evaluar su propia condición.
- **No son excluyentes.** GoF muestra que, cuando la lógica de actualización se vuelve compleja, un `ChangeManager` puede actuar como mediador entre sujetos y observadores (Gamma et al., 1994, p. 337). Aquí la regla de actualización es simple (comparar un precio con un umbral), así que ese intermediario no hace falta y Observer es el patrón principal.

1.1.5. ¿Por qué no usar polling (consulta periódica)?

Criterio	Polling	Observer (push)
Latencia	Hasta un intervalo completo. Si se consulta cada 5 s, el aviso puede llegar 5 s tarde, lo que en trading es inaceptable y contradice "inmediata".	Inmediata.
Carga	clientes × acciones × frecuencia consultas, **cambie o no el precio**. 1000 clientes que siguen 5 acciones, consultando cada 1 s, son 5000 peticiones/s, casi todas inútiles.	Solo hay trabajo cuando cambia el precio.
Eventos perdidos	Si el precio sube y vuelve a bajar entre dos consultas, el cruce **nunca se detecta**.	Cada cambio genera un evento.
Escalabilidad	El costo crece linealmente con los clientes y las acciones aunque no haya actividad.	El costo crece con los eventos relevantes.

Tabla 2: Polling frente a Observer.

Referencias. Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. Los números de página corresponden a la copia del libro en la bibliografía de la asignatura (*Documentacion Patrones*).

1.2. b) Implementación

> *Implementa con 5 acciones, 1000 clientes y umbrales personalizados. Notificaciones por email, SMS y push.*

La implementación sigue el diseño del inciso a: mismas clases y mismas relaciones que el diagrama de clases (figura 1). El código completo está en la carpeta `ejercicio-09-notificaciones-bolsa/codigo/` del repositorio; aquí se muestran solo los fragmentos relevantes.

1.2.1. Cómo ejecutarlo

Requisito: SDK de .NET 10. Desde `ejercicio-09-notificaciones-bolsa/codigo/`:

```
dotnet run --project Bolsa.Notificaciones   # la simulación
dotnet test                                  # las pruebas unitarias (xUnit)
```

Hay que indicar `--project` porque la carpeta tiene dos proyectos (el programa y sus pruebas). No hace falta ningún paso manual: `dotnet` descarga xUnit la primera vez.

1.2.2. Estructura del código

Fichero	Clase del diagrama y rol
`Observer/ISujetoPrecio.cs`	`ISujetoPrecio`: Subject
`Observer/Accion.cs`	`Accion`: ConcreteSubject
`Observer/IObservadorPrecio.cs`	`IObservadorPrecio`: Observer
`Observer/AlertaUmbral.cs`	`AlertaUmbral`: ConcreteObserver; aplica la regla de disparo
`Observer/CambioPrecio.cs`, `Observer/Direccion.cs`	Dato que viaja (modelo push) y dirección del cruce
`Entrega/Cliente.cs`, `Entrega/Notificacion.cs`	Datos del cliente y del aviso
`Entrega/ICanalNotificacion.cs`	`ICanalNotificacion`: Strategy de entrega
`Entrega/Canal{Email,Sms,Push}.cs`	Los 3 canales; el envío se simula en la consola
`Entrega/DespachadorNotificaciones.cs`	Entrega por el canal del cliente y guarda en el repositorio
`Entrega/IRepositorioNotificaciones.cs`, `Entrega/RepositorioMemoria.cs`	Registro de las notificaciones entregadas
`Demos/SimulacionBolsa.cs`	La simulación
`Bolsa.Notificaciones.Tests/`	Pruebas unitarias

Tabla 3: Ficheros del código y su correspondencia con el diagrama.

1.2.3. Regla de disparo

Una alerta **solo notifica cuando el precio cruza su umbral** en su dirección, no en cada cambio mientras el precio siga del otro lado:

- `Sube`: precio anterior $\leq$ umbral **y** precio nuevo $>$ umbral.
- `Baja`: precio anterior $\geq$ umbral **y** precio nuevo $<$ umbral.

Listing 1: `AlertaUmbral`: el observador decide si el cambio cruza su umbral.

```
public void Actualizar(CambioPrecio cambio)
{
    if (!CruzaElUmbral(cambio))
    {
        return;
    }

    string verbo = Direccion == Direccion.Sube ? "subió por encima de" : "bajó por debajo de";
    string mensaje = $"{cambio.Simbolo} {verbo} su umbral {Formato(Umbral)}: " +
                     $"{Formato(cambio.PrecioAnterior)} → {Formato(cambio.PrecioNuevo)}";

    _despachador.Encolar(new Notificacion(Guid.NewGuid(), Cliente, mensaje));
}

private bool CruzaElUmbral(CambioPrecio cambio) => Direccion switch
{
    Direccion.Sube => cambio.PrecioAnterior <= Umbral && cambio.PrecioNuevo > Umbral,
    Direccion.Baja => cambio.PrecioAnterior >= Umbral && cambio.PrecioNuevo < Umbral,
    _ => false,
};
```

Por qué esta regla.

- **Evita avisos repetidos.** Si se notificara siempre que el precio esté por encima del umbral, un cliente recibiría un aviso en cada tick mientras dure la subida. Con el cruce recibe uno solo, en el momento en que ocurre, que es lo que pide "notificaciones inmediatas" cuando "alcanzan precios específicos".
- **Se rearma sola.** Si el precio vuelve al otro lado y cruza otra vez, hay un aviso nuevo: es otro evento. No hace falta guardar estado en la alerta, porque `CambioPrecio` trae el precio anterior y el nuevo (modelo push).
- **Es decisión del observador, no de la acción.** La acción avisa a todas sus alertas en cada cambio, y cada `AlertaUmbral` decide si el cambio cruza *su* umbral. Así la acción no necesita conocer los umbrales.

Si el precio no cambia, `Accion.ActualizarPrecio` no notifica: no hay evento.

Listing 2: `Accion`: el sujeto notifica a todos sus observadores en cada cambio de precio.

```
public void ActualizarPrecio(decimal precioNuevo)
{
    decimal precioAnterior = Precio;
    if (precioNuevo == precioAnterior)
    {
        return;
    }

    Precio = precioNuevo;
    Notificar(new CambioPrecio(Simbolo, precioAnterior, precioNuevo, DateTimeOffset.Now));
}

public void Notificar(CambioPrecio cambio)
{
    foreach (IObservadorPrecio observador in _observadores)
    {
        observador.Actualizar(cambio);
    }
}
```

1.2.4. Datos de la simulación

- **5 acciones**, cada una un Subject independiente con su propia lista de observadores: AAPL (150), MSFT (310), GOOG (140), AMZN (180) y TSLA (250).
- **1000 clientes** generados por código. El canal se reparte por turno (email, SMS, push) y cada cliente se suscribe a **1 o 2 acciones**, alternando entre `Sube` y `Baja`. En total hay **1500 alertas**.
- **Umbrales personalizados:** precio inicial $\pm$ (0,1 % + 0,12 % $\times$ orden de llegada), distinto para cada alerta de la misma acción y dirección. El programa comprueba que no se repite ninguno.
- **10 ticks de precios:** en los ticks 1 a 9 cada precio varía hasta $\pm$1,5 %; el tick 10 simula un evento de mercado con saltos fuertes (+9 %, $-$7 %, +12 %, $-$5 % y +8 %).
- **Desuscripción:** después del tick 8, los primeros 25 clientes se desuscriben de todas sus alertas.
- **Semilla fija** (`20261016`): la simulación es reproducible, da los mismos precios y avisos en cada ejecución.

Listing 3: `Demos/SimulacionBolsa.cs`: cada alerta es un observador con su propio umbral.

```
decimal porcentaje = 0.1m + 0.12m * orden;
decimal umbral = Math.Round(
    direccion == Direccion.Sube
        ? accion.Precio * (1 + porcentaje / 100m)
        : accion.Precio * (1 - porcentaje / 100m),
    2);

AlertaUmbral alerta = new(cliente, umbral, direccion, despachador);
accion.Suscribir(alerta);
```

1.2.5. Resultados

Cada línea `[CANAL → contacto]` es una notificación entregada. Aparece debajo del cambio de precio que la dispara, en el mismo tick:

```
--- Tick 1 ---
  AAPL: 150.00 → 149.44
[EMAIL → cliente0006@correo.cu] AAPL bajó por debajo de su umbral 149.85: 150.00 → 149.44
[SMS → +53 50000016] AAPL bajó por debajo de su umbral 149.67: 150.00 → 149.44
[PUSH → dispositivo-0026] AAPL bajó por debajo de su umbral 149.49: 150.00 → 149.44
  MSFT: 310.00 → 306.49
[PUSH → dispositivo-0002] MSFT bajó por debajo de su umbral 309.69: 310.00 → 306.49
...
  ... 25 clientes se desuscriben (37 alertas quitadas).
```

Resultado	**Valor**
Notificaciones entregadas	417 de 1500 alertas
Clientes notificados	306 de 1000
Avisos por email / SMS / push	142 / 140 / 135
Avisos de AAPL / MSFT / GOOG / AMZN / TSLA	94 / 70 / 106 / 84 / 63
Avisos a los 25 desuscritos: antes / después	30 / **0**
Umbrales repetidos	ninguno

Tabla 4: Resumen de la simulación.

Lo que muestra:

- **Solo se notifica a quien cruzó su umbral:** 417 avisos de 1500 alertas, y solo 306 de los 1000 clientes recibieron alguno. Los umbrales más alejados del precio no se alcanzan.
- **Los tres canales funcionan:** 142 avisos por email, 140 por SMS y 135 por push.
- **La desuscripción funciona:** los 25 clientes desuscritos recibieron 30 avisos antes y **0 después**.
- **El aviso es inmediato:** cada notificación sale en el mismo tick del cruce. En los ticks normales hay entre 11 y 37 avisos; en el tick 10, con los saltos fuertes, 231.

1.2.6. Pruebas unitarias

`dotnet test` ejecuta **11 pruebas, todas correctas**:

Qué comprueba	Criterio
Acercarse sin cruzar no avisa; cruzar al alza avisa una vez y lo guarda	Notifica al cruzar
Lo mismo a la baja	Notifica al cruzar
Subir sin llegar al umbral no avisa	No notifica por debajo
Tres subidas seguidas por encima dan un solo aviso	Regla de disparo
Tras desuscribirse, un nuevo cruce ya no avisa	Desuscripción
Un cambio avisa a todas las alertas cuyo umbral cruza	Uno-a-muchos
Email, SMS y push entregan el mensaje al contacto de su cliente (3 pruebas)	Cada canal recibe su mensaje
El despachador usa el canal de cada cliente	Cada canal recibe su mensaje
Toda notificación entregada queda en el repositorio	—

Tabla 5: Pruebas unitarias (xUnit).

Listing 4: Prueba de la desuscripción.

```
[Fact]
public void NoNotifica_DespuesDeDesuscribirse()
{
    (Accion accion, AlertaUmbral alerta, CanalDePrueba canal, _) =
        CrearEscenario(150.00m, 152.00m, Direccion.Sube);

    accion.ActualizarPrecio(153.00m);
    Assert.Single(canal.Enviadas);

    accion.Desuscribir(alerta);

    accion.ActualizarPrecio(151.00m); // baja y vuelve a cruzar el umbral al alza
    accion.ActualizarPrecio(152.50m);

    Assert.Single(canal.Enviadas); // solo el aviso anterior a la desuscripción
}
```

1.2.7. Decisiones de esta implementación

- **El despachador entrega de forma síncrona.** `Encolar` deja la notificación en la cola (`Channel<Notificacion>`) y la procesa al momento. Así la simulación es determinista y las pruebas no dependen de hilos. El inciso c convierte ese envío en asíncrono sin cambiar su lógica: las alertas solo encolan.
- **El repositorio está en memoria.** `RepositorioMemoria` registra las notificaciones entregadas. La restricción (E9-R) añadirá una implementación persistente y los reintentos; el despachador no cambia, porque solo conoce `IRepositorioNotificaciones`.
- **El envío es simulado:** cada canal escribe el aviso en la consola con su tipo y el contacto del cliente.

1.3. c) Análisis

> *¿Qué pasa si un cliente no puede ser notificado? ¿Cómo evitar bloquear el hilo principal? ¿Observer es síncrono o asíncrono?*

Las tres preguntas se responden sobre el código del inciso b. Nada de lo que se afirma aquí se queda en la teoría: cada punto tiene su demostración ejecutable y su prueba unitaria.

```
cd codigo
dotnet run --project Bolsa.Notificaciones -- asincrono   # la demostración de este inciso
dotnet test                                              # 20 pruebas, 9 de este inciso
```

Pregunta	Respuesta
¿Qué pasa si un cliente no puede ser notificado?	En el Observer clásico, **la excepción corta el bucle de notificación** y los suscriptores que vienen detrás se quedan sin aviso. Se arregla con dos cosas distintas: aislar a cada observador (para no perder a los demás) y una cola con reintentos (para no perder al que falló).
¿Cómo evitar bloquear el hilo principal?	Separando *detectar* el cruce de *enviar* el aviso: la alerta solo encola y el envío lo hacen consumidores en segundo plano. Medido: el cambio de precio pasa de **2002,8 ms a 0,1 ms** con un canal de 2 s.
¿Observer es síncrono o asíncrono?	**Síncrono por defecto.** `Notificar` llama a `Actualizar` como una llamada de método normal, en el hilo de quien cambió el estado. Puede hacerse asíncrono, pero el patrón no lo es: lo asíncrono es lo que el observador hace con el aviso.

Tabla 6: Las respuestas en corto.

1.3.1. ¿Qué pasa si un cliente no puede ser notificado?

El problema: el fallo de un cliente deja sin aviso a los demás. El Observer de GoF recorre la lista de observadores y llama a `Update` sin ninguna protección (Gamma et al., 1994, p. 335):

Listing 5: El bucle de notificación del libro, sin red.

```
void Subject::Notify () {
    ListIterator<Observer*> i(_observers);
    for (i.First(); !i.IsDone(); i.Next()) {
        i.CurrentItem()->Update(this);
    }
}
```

El `Notificar` del inciso b era exactamente eso. Si un cliente no se puede notificar —un correo que ya no existe, un teléfono dado de baja, la pasarela de SMS caída—, la excepción recorre toda la cadena hacia arriba:

```
CanalSms.EnviarAsync  →  lanza
  DespachadorNotificaciones.Encolar   (entrega síncrona: la relanza)
    AlertaUmbral.Actualizar
      Accion.Notificar                 <- el foreach se interrumpe aquí
        Accion.ActualizarPrecio        <- y la excepción sale del cambio de precio
```

Las consecuencias concretas en este problema son graves:

- **Un solo cliente silencia a una multitud.** Las alertas se recorren en el orden en que se suscribieron, así que un número de teléfono dado de baja deja sin aviso a todos los que se suscribieron después de él. En la simulación del inciso b, el tick 10 reparte **231 avisos**: si el segundo falla, los otros 229 no salen.
- **El sujeto queda a medias.** La excepción sale de `ActualizarPrecio`, de modo que un problema de *entrega* acaba rompiendo la *cotización*. Son dos responsabilidades que no deberían compartir destino.
- **El fallo es silencioso para quien importa.** Nadie registra que ese cliente no recibió su aviso, y la notificación se pierde sin dejar rastro.

Es el riesgo que GoF ya señala en las consecuencias del patrón: como los observadores no se conocen entre sí, "una operación aparentemente inocua sobre el sujeto puede provocar una cascada de actualizaciones" cuyo costo el sujeto no controla (Gamma et al., 1994, p. 330). Aquí la cascada no solo es costosa: es frágil.

Listing 6: Demostración: el Subject clásico con un canal caído en el medio.

```
--- 1a. Observer clásico: un fallo corta el bucle ---
[EMAIL → cliente0001@correo.cu] AAPL subió por encima de su umbral 151.00: 150.00 → 155.00
  ActualizarPrecio lanzó InvalidOperationException: El proveedor rechazó el aviso de +53 50000002
  -> Solo se entregó el aviso del primer cliente. El tercero se quedó sin avisar.
```

Primer arreglo: aislar cada observador. Cada `Actualizar` se envuelve en su propio `try/catch`, de modo que el recorrido continúa y el fallo se registra:

Listing 7: `Accion`: el fallo de un observador ya no corta el bucle.

```
public void Notificar(CambioPrecio cambio)
{
    foreach (IObservadorPrecio observador in _observadores)
    {
        try
        {
            observador.Actualizar(cambio);
        }
        catch (Exception error)
        {
            _registro.Registrar(
                nameof(Accion),
                $"{Simbolo}: un observador falló al actualizarse y se omitió",
                error);
        }
    }
}
```

Dos decisiones acompañan a ese `catch`:

- **Se captura `Exception`, lo más general posible.** Es lo correcto en un *límite de aislamiento*: el sujeto no puede saber qué tipos de excepción lanzan los canales de terceros, y cualquiera de ellos tiene el mismo efecto indeseado.
- **No se silencia nunca.** `IRegistroFallos` existe justo para eso, y si no se le pasa ninguno, `Accion` usa uno que escribe en la consola. Un `catch` vacío cambiaría un problema visible por uno invisible.

```
--- 1b. Con aislamiento: el fallo no contagia a los demás ---
[EMAIL → cliente0001@correo.cu] AAPL subió por encima de su umbral 151.00: 150.00 → 155.00
[FALLO Accion] AAPL: un observador falló al actualizarse y se omitió (InvalidOperationException: ...)
[PUSH → dispositivo-0003] AAPL subió por encima de su umbral 151.00: 150.00 → 155.00
  -> Clientes 1 y 3 avisados; 1 fallo registrado (total: 1).
```

Pero esto solo resuelve la mitad del problema. Los demás clientes ya reciben su aviso; el del cliente que falló se perdió, porque con entrega síncrona no hay dónde guardarlo ni cuándo volver a intentarlo.

Segundo arreglo: la notificación fallida no se pierde. Para eso hace falta la cola. `DespachadorAsincrono` aísla **cada envío**, no solo cada observador, y en lugar de descartar la notificación la vuelve a encolar:

Listing 8: `DespachadorAsincrono`: aislamiento por envío, reintento y dead letter.

```
private async Task IntentarEnviarAsync(EnvioPendiente envio)
{
    try
    {
        await envio.Notificacion.Cliente.Canal.EnviarAsync(envio.Notificacion);
        _repositorio.Guardar(envio.Notificacion);
        Interlocked.Increment(ref _entregadas);
        Interlocked.Decrement(ref _enProceso);
    }
    catch (Exception error)
    {
        Interlocked.Increment(ref _fallosDeEnvio);
        int intentos = envio.Intentos + 1;
        _registro.Registrar(nameof(DespachadorAsincrono), $"intento {intentos}/{MaxIntentos}",
    error);

        if (intentos >= MaxIntentos)
        {
            DarPorPerdida(envio.Notificacion);   // dead letter
            return;
        }

        Reprogramar(envio with { Intentos = intentos });
    }
}
```

La espera entre intentos se duplica cada vez (50 ms, 100 ms, 200 ms…) y **no bloquea al consumidor**: se programa con `Task.Delay` y el consumidor sigue atendiendo el resto de la cola.

Desenlace	Cuándo	Qué queda
Entregada	el canal confirma	se guarda en `IRepositorioNotificaciones`
Reintento	el canal falla y quedan intentos	vuelve a la cola tras la espera, con el contador subido
Fallo definitivo (*dead letter*)	se agotaron los `MaxIntentos`	queda en `FallosDefinitivos` para no perderla de vista

Tabla 7: Los tres desenlaces posibles de un envío.

Listing 9: Un proveedor que se recupera, uno que no responde nunca y uno normal.

```
--- 2. Reintentos con espera creciente y fallo definitivo ---
[FALLO DespachadorAsincrono] intento 1/3 del aviso de +53 50000005 (...)
[FALLO DespachadorAsincrono] intento 1/3 del aviso de cliente0004@correo.cu (...)
[PUSH → dispositivo-0006] AAPL subió por encima de su umbral 151.00: 150.00 → 155.00
[FALLO DespachadorAsincrono] intento 2/3 del aviso de +53 50000005 (...)
[FALLO DespachadorAsincrono] intento 2/3 del aviso de cliente0004@correo.cu (...)
[FALLO DespachadorAsincrono] intento 3/3 del aviso de +53 50000005 (...)
[EMAIL → cliente0004@correo.cu] AAPL subió por encima de su umbral 151.00: 150.00 → 155.00
  -> Entregadas: 2 de 3 (C4 llegó en el intento 3).
  -> Intentos fallidos: 5; reintentos programados: 4.
  -> Fallo definitivo (dead letter): 1 tras 3 intentos.
```

El cliente intermitente **sí recibió su aviso**, en el tercer intento: eso es lo que un reintento compra. El cliente imposible acabó en fallo definitivo después de 3 intentos, registrado y localizable, no perdido en silencio.

Lo que este inciso todavía no resuelve. Aquí la cola, los reintentos y los fallos definitivos viven **en memoria**: si el proceso se cae, lo que estuviera en vuelo se pierde. Esa es exactamente

la pieza que añade la restricción del ejercicio: persistir la notificación *antes* de enviarla y marcarla como entregada solo cuando el canal confirma (patrón *outbox*), para garantizar la entrega *at-least-once* incluso entre reinicios. El diseño ya está preparado: `DespachadorAsincrono` solo conoce `IRepositorioNotificaciones`, así que cambiar `RepositorioMemoria` por uno persistente no toca al despachador.

1.3.2. ¿Cómo evitar bloquear el hilo principal?

El problema: el sujeto paga la suma de todos los envíos. Con entrega síncrona, `ActualizarPrecio` no termina hasta que **el último canal** responde. Las cuentas de este problema concreto lo dejan claro: en el tick 10 de la simulación del inciso b se reparten **231 avisos**, y con un proveedor realista de 200 ms por envío:

	Entrega síncrona	**Entrega asíncrona (4 consumidores)**
Tiempo del cambio de precio	$231 \times 200\,\text{ms} = \mathbf{46{,}2\,s}$	lo que cuesta encolar: $< \mathbf{1\,ms}$
Qué ocurre con el siguiente tick	llega mientras el anterior sigue enviando; la cotización se atrasa sin límite	se atiende de inmediato

Tabla 8: El costo de bloquear el hilo del sujeto.

46,2 s es una estimación del problema: 231 avisos del tick 10 a 200 ms cada uno. No es una medida de esta máquina: es el tiempo que la entrega síncrona le haría pagar al hilo de cotización antes de poder seguir.

Una plataforma de trading no puede detener la cotización durante 46 segundos para mandar SMS. Y el problema no es solo de rendimiento: es de **responsabilidades**. Que `Accion` se bloquee por un proveedor de SMS significa que el sujeto quedó acoplado al costo de sus observadores, justo lo que GoF advierte que el patrón no controla (Gamma et al., 1994, p. 330).

La solución: una cola entre detectar y enviar. Entre el observador que detecta el cruce y el canal que entrega el aviso se mete una cola acotada, y el envío pasa a hacerlo un grupo de consumidores en segundo plano.

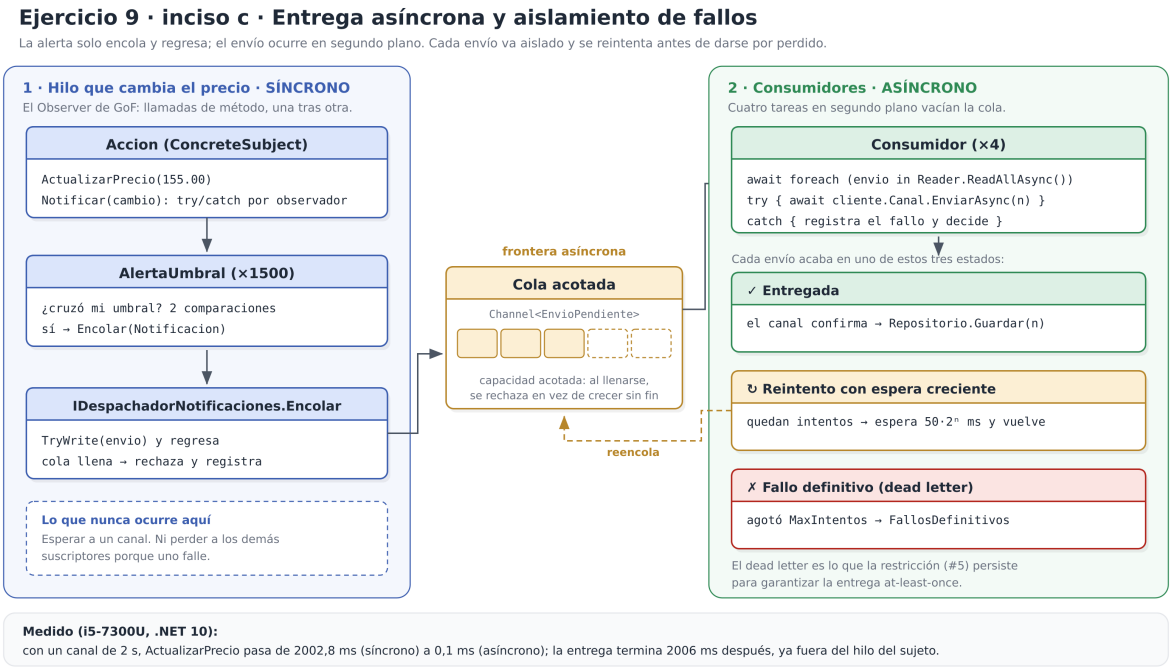


Figura 2: La alerta solo encola; el envío ocurre en segundo plano, aislado y con reintentos.

La alerta sigue haciendo lo mismo que en el inciso b —decidir si el precio cruzó su umbral y encolar—, pero ahora **`Encolar` solo escribe en la cola y regresa**:

Listing 10: Encolar no espera nunca; los consumidores vacían la cola en segundo plano.

```
public void Encolar(Notificacion notificacion)
{
    Interlocked.Increment(ref _enProceso);
    if (_cola.Writer.TryWrite(new EnvioPendiente(notificacion, Intentos: 0)))
    {
        return;
    }

    // Cola llena: esperar aquí sería volver a bloquear al sujeto.
    Interlocked.Decrement(ref _enProceso);
    Interlocked.Increment(ref _rechazadas);
    _registro.Registrar(nameof(DespachadorAsincrono), $"cola llena ({Capacidad})");
}

private async Task ConsumirAsync()
{
    await foreach (EnvioPendiente envio in _cola.Reader.ReadAllAsync())
    {
        await IntentarEnviarAsync(envio);
    }
}
```

Decisión	Por qué
`System.Threading.Channels` y no una `Queue` con `lock`	`Channel<T>` es una cola productor-consumidor pensada para esto: los consumidores esperan con `await` sin consumir CPU, y `ReadAllAsync` reparte los elementos entre varios lectores sin sincronizar a mano.
Cola acotada (`CreateBounded`) y no ilimitada	Una cola ilimitada convierte una caída del proveedor en memoria creciente hasta agotar el proceso. Acotada, el sistema tiene que decidir qué hacer cuando se llena; esa decisión es el *backpressure* del inciso d.
`TryWrite` y no `WriteAsync`	`WriteAsync` esperaría cuando la cola está llena, y esa espera la pagaría otra vez el hilo del cambio de precio. Con `TryWrite` el sujeto nunca espera: en el peor caso la notificación se rechaza y queda registrada.

Tabla 9: Las tres decisiones de diseño de la cola.

La medición. Medida con `Stopwatch`, con canales silenciosos (escribir en la consola cuesta más que el envío simulado y falsearía el resultado):

Escena	Síncrona	Asíncrona	En segundo plano
1 canal lento de 2 s + 2 rápidos	2002,8 ms	**0,1 ms**	2006,3 ms
4 canales lentos de 500 ms, 4 consumidores	2000,8 ms	**0,1 ms**	502,0 ms

Tabla 10: Lo que tarda el cambio de precio. Máquina: Intel Core i5-7300U a 2,60 GHz (2 núcleos, 4 hilos), 15 GiB de RAM, Linux, .NET 10.0.113.

1. **El cambio de precio regresa enseguida** aunque el canal tarde 2 s: 0,1 ms frente a 2002,8 ms, cuatro órdenes de magnitud. El trabajo no desapareció, se movió de hilo.
2. **Varios consumidores solapan los envíos:** cuatro canales de 500 ms tardan 502 ms en total en vez de 2000 ms. Esto es paralelismo, no un cambio de complejidad; de eso se ocupa el inciso d.

Lo que cuesta desacoplar el envío. No es gratis, y conviene decirlo:

- **El orden de llegada deja de estar garantizado** (se demuestra más abajo).
- **Aparece concurrencia.** Varios consumidores escriben a la vez en el repositorio y en los contadores: de ahí el `lock` de `RepositorioMemoria` y los `Interlocked` del despachador.
- **El aviso puede llegar tarde o duplicado.** Si el proceso se reinicia entre el envío y la confirmación, la notificación se reenvía: eso es *at-least-once*, y es el tema de la restricción.
- **La saturación se hace visible.** Con cola acotada hay que decidir qué pasa cuando los cambios de precio llegan más rápido de lo que se puede notificar.

1.3.3. ¿Observer es síncrono o asíncrono?

El Observer de GoF es síncrono. Síncrono, y por una razón estructural, no por una elección de implementación. En el código del patrón, `Notify` recorre los observadores y llama a `Update` como una **llamada de método ordinaria**: no vuelve de esa llamada hasta que el observador terminó, y todo ocurre en el hilo de quien cambió el estado del sujeto (Gamma et al., 1994, p. 335). No hay en el patrón ni cola, ni hilo, ni promesa.

De hecho, el patrón *depende* de eso en un punto: GoF insiste en que el sujeto debe estar en un estado coherente **antes** de llamar a `Notify` (Gamma et al., 1994, p. 331). Esa recomendación tiene sentido precisamente porque el observador va a leer el sujeto en ese mismo instante.

Mecanismo	**¿Síncrono?**
`Notificar` / `Actualizar` escritos a mano (nuestro caso)	Sí: llamada de método en el hilo del sujeto.
`event` + `PrecioCambiado?.Invoke(...)`	Sí: invoca a los suscriptores uno tras otro y en el mismo hilo; si uno lanza, los siguientes no se ejecutan (el mismo problema de la primera pregunta).
`IObservable<T>` / `IObserver<T>` (Rx.NET)	`OnNext` también se llama en el hilo del productor, salvo que se interponga explícitamente un planificador (`ObserveOn`, `SubscribeOn`).

Tabla 11: En C# los mecanismos del lenguaje también son síncronos.

Puede hacerse asíncrono, pero lo asíncrono no es el patrón. La distinción importa para no decir algo falso: lo que se vuelve asíncrono es el *envío*, no la notificación.

Paso	**¿Síncrono o asíncrono?**
`Accion.ActualizarPrecio` → `Notificar`	Síncrono.
`AlertaUmbral.Actualizar` (evalúa el cruce)	Síncrono, en el hilo del sujeto. Es una comparación de dos `decimal`: barata.
`IDespachadorNotificaciones.Encolar`	Síncrono **pero no bloqueante**: escribe en la cola y regresa.
`ICanalNotificacion.EnviarAsync`	**Asíncrono**, en los consumidores de la cola.

Tabla 12: Dónde está exactamente la frontera asíncrona.

Es decir: el Observer sigue siendo el Observer síncrono de GoF, y la frontera asíncrona se pone **dentro del observador**, en lo que este hace con el aviso. Esa es la razón de que `AlertaUmbral` no haya cambiado nada de su lógica entre el inciso b y este: depende de `IDespachadorNotificaciones`, y cambiar la implementación síncrona por la asíncrona no la afecta.

Las consecuencias de hacerlo asíncrono. El orden de llegada ya no está garantizado. Con tres clientes suscritos en el orden 1, 2, 3 y canales de 300, 100 y 200 ms:

```
--- 3c. El orden de llegada ya no está garantizado ---
  Síncrono (orden de suscripción):
[PUSH → cliente1 (300 ms)] ...
[PUSH → cliente2 (100 ms)] ...
[PUSH → cliente3 (200 ms)] ...
  Asíncrono (orden de respuesta de cada canal):
[PUSH → cliente2 (100 ms)] ...
[PUSH → cliente3 (200 ms)] ...
[PUSH → cliente1 (300 ms)] ...
```

Problema	Por qué aparece	Cómo se trata aquí
Orden entre avisos	varios consumidores avanzan a distinta velocidad	No importa en este problema: cada aviso es independiente y lleva su propio precio. Si hiciera falta orden por cliente, habría que repartir por `Cliente.Id`.
Estado obsoleto	el precio puede cambiar otra vez antes de que salga el aviso	Resuelto por el modelo push del inciso a: `CambioPrecio` y `Notificacion` llevan una copia de los precios, así que el consumidor nunca lee un valor más nuevo que el del evento.
Carreras de datos	varios hilos tocan el repositorio y los contadores	`lock` en `RepositorioMemoria`, `Interlocked` en los contadores del despachador.
Duplicados	un reintento puede enviar dos veces si el primer envío llegó pero la confirmación se perdió	`Notificacion` ya tiene `Id` único (clave de idempotencia). Es la base del *at-least-once* de la restricción.
Fallos fuera de la pila original	la excepción ya no sube al hilo del sujeto	Por eso cada envío tiene su `try/catch` y su registro: en segundo plano, una excepción no capturada no se ve.
Saturación	los eventos llegan más rápido de lo que se envía	Cola acotada y rechazo registrado; alternativas en el inciso d.

Tabla 13: Lo que hay que vigilar cuando la entrega deja de ser síncrona.

1.3.4. Cambios respecto al código del inciso b

El diseño del inciso a no cambia: los roles del patrón son los mismos. Lo que se añade está en la zona de entrega.

Fichero	Qué es
`Entrega/IDespachadorNotificaciones.cs`	**Nuevo.** Lo único que la alerta necesita: `Encolar`.
`Entrega/DespachadorAsincrono.cs`	**Nuevo.** Cola acotada, consumidores, reintentos y dead letter.
`Entrega/EnvioPendiente.cs`	**Nuevo.** La notificación más el número de intentos.
`Entrega/CanalLento.cs`, `Entrega/CanalQueFalla.cs`	**Nuevos.** Decoradores para simular un proveedor lento y uno caído.
`Entrega/CanalSilencioso.cs`	**Nuevo.** Canal que solo cuenta, para no falsear las mediciones.
`Diagnostico/IRegistroFallos.cs`, `Diagnostico/RegistroFallos.cs`	**Nuevos.** Donde se anota cada fallo aislado.
`Observer/Accion.cs`	**Modificado.** `Notificar` aísla a cada observador.
`Observer/AlertaUmbral.cs`	**Modificado.** Depende de la interfaz, no de la clase concreta. Su lógica no cambió.
`Entrega/DespachadorNotificaciones.cs`	**Modificado.** Implementa la nueva interfaz; su comportamiento es el mismo.
`Demos/SimulacionBolsa.cs`	**Movido** desde `Program.cs`, sin cambios de lógica.
`Demos/AccionSinAislamiento.cs`	**Nuevo.** El Subject clásico, solo para la demostración.
`Demos/DemoAsincrona.cs`	**Nuevo.** La demostración de este inciso.

Tabla 14: Ficheros nuevos y modificados.

La simulación del inciso b sigue dando exactamente los mismos resultados (417 avisos, 306 clientes, 0 avisos tras la desuscripción).

1.3.5. Pruebas unitarias

`dotnet test` ejecuta **20 pruebas, todas correctas**; 9 son de este inciso:

Prueba	Qué comprueba
`ElBucleClasicoSeCorta_Cuando-UnObservadorFalla`	Sobre el Subject clásico: el primero recibe, el último no, y la excepción sale de `ActualizarPrecio`
`UnClienteNoNotificableNoImpide-QueLosDemasReciban`	Misma escena con `Accion`: los dos clientes sanos reciben y el fallo queda registrado
`ElPrecioSeActualizaAunque-UnObservadorFalle`	Un fallo de entrega no corrompe la cotización
`ReintentaYEntrega_Cuando-ElCanalSeRecupera`	Canal que falla 2 veces: 2 reintentos, 3 intentos, entregada
`DaPorPerdidaLaNotificacion_-TrasAgotarLosIntentos`	Canal siempre caído: 3 intentos y dead letter; no se guarda como entregada
`UnCanalCaidoNoImpideLa-EntregaDeLosDemasClientes`	Aislamiento por envío, no solo por observador
`EntregaEnSegundoPlanoTodasLas-NotificacionesEncoladas`	50 notificaciones encoladas se entregan todas
`ElCambioDePrecioNoEspera-AlCanalLento`	Con un canal de 1 s, `ActualizarPrecio` regresa en menos de 500 ms y nada se ha entregado todavía; tras vaciar la cola, sí
`RechazaSinBloquear_Cuando-LaColaEstaLlena`	100 avisos con capacidad 1: hay rechazos, encolar no espera, y cada aviso acaba entregado o rechazado

Tabla 15: Las pruebas del inciso c.

1.4. d) Rendimiento

> *Evalúa el rendimiento con 1M de clientes. ¿Cómo cambia el orden de complejidad? Propón Observer con Thread Pool o Reactive Streams.*

Este inciso mide el código del inciso b con la entrega asíncrona del inciso c, propone una alternativa indexada y la mide junto a la versión ingenua.

```
cd codigo
dotnet run -c Release --project Bolsa.Notificaciones -- rendimiento        # hasta 1 000 000
dotnet run -c Release --project Bolsa.Notificaciones -- rendimiento 100000 # más rápido
dotnet test                                                                # 27 pruebas
```

Importante: `-c Release`. En `Debug` el compilador no optimiza y los tiempos no significan nada.

Pregunta	Respuesta
¿Cómo se comporta con 1M de clientes?	Un solo cambio de precio tarda **112,3 ms** recorriendo las alertas una por una, y ocupa **244 MB**. Con 5 acciones cotizando cada segundo, la detección sola consume más de medio segundo por segundo: el sistema no llega.
¿Cómo cambia el orden de complejidad?	La versión ingenua es $\Theta(n)$ por cambio de precio: revisa los n umbrales aunque solo haya que avisar a k clientes. Ordenando los umbrales, los cruzados son un **tramo contiguo** que se localiza con dos búsquedas binarias: $O(\log n + k)$. Medido: 112,3 ms → **0,154 ms** (731×).
¿Thread Pool o Reactive Streams?	**Las dos cosas, en sitios distintos:** un pool de consumidores para repartir los k envíos (medido: 6 409 avisos/s con 8 consumidores frente a 792 con uno), y una **cola acotada** (`System.Threading.Channels`) para el *backpressure*. Rx.NET no sirve para lo segundo, y se explica por qué.

Tabla 16: Las respuestas en corto.

1.4.1. Cómo se midió

- `Stopwatch`, no BenchmarkDotNet: la unidad que interesa no es una función aislada sino *un cambio de precio sobre una estructura de n suscripciones*, que hay que construir antes y que ocupa cientos de megabytes. BenchmarkDotNet habría dado más rigor estadístico a costa de no poder medir eso.
- **Mediana de 11 ticks**, descartando 3 de calentamiento. La mediana y no la media: una pausa del recolector de basura en un solo tick desplazaría la media.
- **Canales simulados sin E/S real** (`CanalSilencioso`, que solo cuenta). Escribir en la consola cuesta más que el envío simulado y falsearía la medición.
- **Las dos fases se miden por separado**, porque solo una cambia de orden: *detectar* a quién avisar y *entregar* los avisos.
- **Semillas y umbrales deterministas:** el benchmark es reproducible.

Máquina: Intel Core i5-7300U a 2,60 GHz (2 núcleos, 4 hilos), 15 GiB de RAM, Garuda Linux x64 (kernel 7.2.11-zen), .NET SDK 10.0.113 / runtime 10.0.13, GC de estación de trabajo. Commit medido: `43820e0`.

Los dos escenarios. El resultado depende por completo de **cuántos clientes hay que avisar de verdad** en cada tick, así que se miden los dos extremos.

Escenario	Cómo se reparten los umbrales	k por tick
A · dispersos	un umbral por centavo (densidad fija); con más clientes, la banda de precios cubierta es más ancha	**constante**: 50, no importa cuánto crezca n
B · concentrados	los n umbrales repartidos en una banda fija de 30 $ alrededor del precio	**proporcional a** n: $n/100$

Tabla 17: Ninguno es «el realista»: el real está en medio, y por eso interesa ver los dos.

1.4.2. Resultados

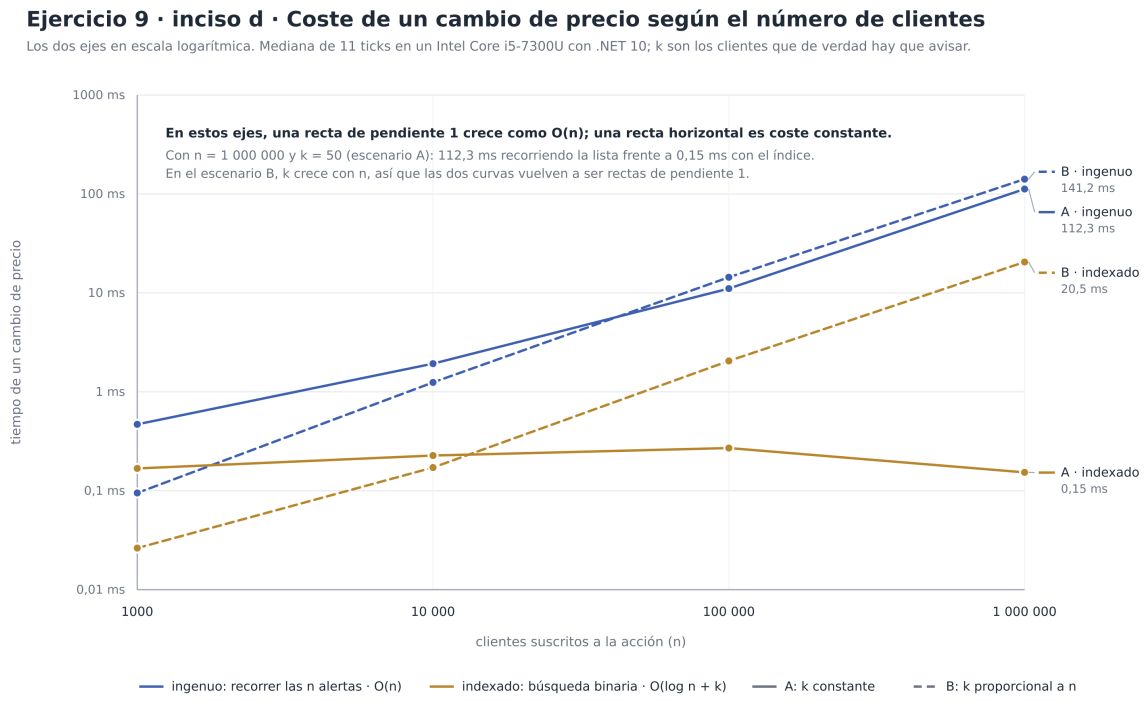


Figura 3: El recorrido ingenuo es una recta de pendiente 1 (crece como n); el índice con k constante es una recta horizontal.

n	k	Ingenuo	Indexado	Mejora	Memoria	Construcción
1 000	50	0,469 ms	0,168 ms	2,8×	0,2 / 0,2 MB	2 / 1 ms
10 000	50	1,925 ms	0,227 ms	8,5×	2,4 / 1,8 MB	40 / 47 ms
100 000	50	11,068 ms	0,270 ms	41,0×	24,4 / 17,7 MB	577 / 169 ms
1 000 000	50	**112,275 ms**	**0,154 ms**	**731,4×**	244,1 / 174,8 MB	2 652 / 1 108 ms

Tabla 18: Escenario A: umbrales dispersos ($k = 50$ constante). Memoria y construcción: ingenuo / indexado.

El ingenuo multiplica su tiempo por 10 cada vez que n se multiplica por 10: eso *es* $\Theta(n)$. El indexado se queda entre 0,15 y 0,27 ms en todo el rango. Las diferencias entre esos cuatro valores son ruido de medición, no crecimiento; lo que domina ahí no es el $\log n$ (20 comparaciones) sino crear los 50 mensajes.

n	k	Ingenuo	Indexado	Mejora	Memoria	Construcción
1 000	10	0,095 ms	0,026 ms	3,6×	0,2 / 0,2 MB	1 / 1 ms
10 000	100	1,245 ms	0,172 ms	7,3×	2,4 / 1,7 MB	12 / 5 ms
100 000	1 000	14,383 ms	2,054 ms	7,0×	24,4 / 16,8 MB	252 / 63 ms
1 000 000	10 000	**141,246 ms**	**20,510 ms**	**6,9×**	244,1 / 167,8 MB	2 879 / 1 015 ms

Tabla 19: Escenario B: umbrales concentrados ($k = n/100$). La mejora se estanca en $\sim$7×.

Aquí la mejora **no crece con** n. No es un fallo del índice: cuando k es proporcional a n, el término k domina y $O(\log n + k)$ *también* es $\Theta(n)$. El índice sigue ganando, pero por una constante (se ahorra tocar los 990 000 clientes que no cruzaron), no por un orden.

La lección del par de tablas: el índice no "hace rápido" el sistema. Elimina el trabajo inútil —mirar umbrales que no se cruzaron— y deja solo el trabajo inevitable: atender a los k clientes que sí hay que avisar.

1.4.3. ¿Cómo cambia el orden de complejidad?

La versión ingenua: $\Theta(n)$ por cambio de precio. Por cada cambio de precio de una acción se recorren **todos** sus observadores, se cruce su umbral o no. Si esa acción tiene n suscripciones, el costo es $\Theta(n)$, y sumando el trabajo de los k que sí cruzan, $\Theta(n + k) = \Theta(n)$, porque $k \leq n$.

Con 1 000 000 de clientes y un tick por segundo en 5 acciones, eso es **5 000 000 de comprobaciones por segundo** para, en el escenario A, avisar a 250 personas. El 99,99 % del trabajo se tira.

La versión indexada: $O(\log n + k)$. La clave está en que la regla de disparo del inciso b es un **predicado de intervalo**: `Sube` notifica si el umbral está en $[\text{anterior}, \text{nuevo})$, y `Baja` si está en $(\text{nuevo}, \text{anterior}]$. Si los umbrales están **ordenados**, los que cumplen esa condición forman un **tramo contiguo** del array, y localizar sus dos extremos es una búsqueda binaria cada uno.

Listing 11: `GestorUmbralesIndexado`: dos búsquedas binarias y el recorrido del tramo.

```
public void Actualizar(CambioPrecio cambio)
{
    if (cambio.PrecioNuevo > cambio.PrecioAnterior)
    {
        Avisar(
            _umbralesSube,
            _clientesSube,
            PrimeroMayorOIgual(_umbralesSube, cambio.PrecioAnterior),  // O(log n)
            PrimeroMayorOIgual(_umbralesSube, cambio.PrecioNuevo),     // O(log n)
            Direccion.Sube,
            cambio);
    }
    else if (cambio.PrecioNuevo < cambio.PrecioAnterior)
    {
        // Una bajada solo puede cruzar umbrales "Baja", y al revés.
        Avisar(_umbralesBaja, _clientesBaja,
            PrimeroMayor(_umbralesBaja, cambio.PrecioNuevo),
            PrimeroMayor(_umbralesBaja, cambio.PrecioAnterior),
            Direccion.Baja, cambio);
    }
}
```

Esto no rompe el patrón: el índice es un ChangeManager. Podría parecer que indexar los umbrales obliga al sujeto a conocer a los clientes, que es justo lo que el inciso a evitaba. No es así: el índice vive **dentro de un único observador**.

```
var accion = new Accion("AAPL", 150.00m);
accion.Suscribir(GestorUmbralesIndexado.Construir(suscripciones, despachador));
```

`Accion` sigue siendo el Subject, sigue teniendo una lista de `IObservadorPrecio` y **sigue sin saber nada de umbrales ni de clientes**: ahora tiene un observador en lugar de un millón. Es exactamente el `ChangeManager` que GoF propone cuando la lógica de actualización se vuelve costosa (Gamma et al., 1994, p. 333), y que el inciso a ya había mencionado como la puerta por la que un mediador puede entrar sin desplazar a Observer.

Lo que *sí* se pierde es que cada suscripción deje de ser un objeto con su propia lógica: la condición ya no la evalúa el observador, la resuelve el índice. Es el precio del cambio, y solo compensa cuando n es grande: con 1 000 clientes la mejora es 2,8×; con 1 000 000, 731×.

Como el índice agrupa a todos los clientes de una acción bajo un único observador, `Avisar` recorre el tramo de una sentada: si `Encolar` lanzara a mitad del tramo, los clientes restantes se quedarían sin aviso. Ese es el riesgo de concentrar el envío en un solo observador. El `Encolar` actual de `DespachadorAsincrono` no lanza (escribe con `TryWrite` y, si la cola está llena, rechaza y registra), así que el riesgo no se materializa hoy, pero es una consecuencia estructural de agrupar bajo un único observador.

El límite inferior. Ninguna estructura de datos puede bajar de $\Omega(k)$, porque hay que crear y entregar k notificaciones. $O(\log n + k)$ es óptimo en ese sentido: $\log n$ es el costo de *encontrar* a quién avisar y k es el trabajo que el problema exige. El escenario B es lo que se ve cuando se alcanza ese límite.

Memoria. Las dos versiones son $\Theta(n)$: hay que guardar n suscripciones. Con 1 000 000 de clientes, 244,1 MB frente a 174,8 MB (29 % menos). El índice ahorra el objeto `AlertaUmbral` por suscripción y los nodos del `ImmutableList` del sujeto, y los sustituye por dos arrays planos (`decimal[]` y `Cliente[]`), que además recorren memoria contigua. Lo que domina en ambos casos son los propios clientes (identificador y contacto), que son irreducibles.

Altas y bajas: lo que el índice empeora. La contrapartida honesta de la propuesta está en las operaciones de suscripción: lo que el recorrido gana, el mantenimiento del orden lo paga.

Operación	**Ingenuo (`ImmutableList`)**	**Indexado (array ordenado)**
Suscribir	$O(\log n)$ con copia de camino	$O(n)$: hay que insertar en el array
Desuscribir	$O(\log n)$	$O(n)$
Notificar	$\Theta(n)$	$O(\log n + k)$
Construir desde cero	2 652 ms para 1M	1 108 ms para 1M

Tabla 20: La contrapartida honesta de la propuesta.

El prototipo indexado solo se construye de una vez (`GestorUmbralesIndexado.Construir`): no ofrece `Suscribir` ni `Desuscribir` dinámicas sobre el índice ya construido. Esa es precisamente la contrapartida que cuantifica la tabla anterior —mantener el orden cuesta $O(n)$ por alta o baja en el array ordenado, frente a $O(\log n)$ del `ImmutableList` ingenuo—; si las suscripciones cambian a menudo, conviene la estructura ordenada con inserción logarítmica que se comenta a continuación.

Si las altas y bajas fueran tan frecuentes como los ticks, el array ordenado sería mala idea. La solución es una **estructura ordenada con inserción logarítmica** (`SortedSet`, un árbol balanceado o una *skip list*): mantiene $O(\log n)$ para suscribir y desuscribir, y el recorrido del tramo sigue siendo $O(\log n + k)$. Aquí se usa un array porque en este problema los precios cambian muchísimo más a menudo que las suscripciones, y el array da la mejor constante en el recorrido.

1.4.4. El paralelismo reduce el tiempo de reloj, no el orden

Escenario B, $n = 1\,000\,000$**,** $k = 10\,000$	**Tiempo por tick**
Indexado secuencial	21,668 ms
Indexado en paralelo (4 hilos)	12,233 ms
Aceleración	**1,77**×

Tabla 21: Repartir el tramo encontrado con `Parallel.For`.

El trabajo total no bajó: se repartió. Con p procesadores, $O(\log n+k)$ pasa a $O(\log n+k/p)$ de tiempo de reloj, pero el *trabajo* sigue siendo $O(\log n + k)$. Como p es una constante de la máquina (aquí 4 hilos sobre 2 núcleos físicos), el orden de complejidad **no cambia**: sigue siendo lineal en k.

La aceleración de 1,77× sobre 4 hilos lógicos es lo esperable: son 2 núcleos físicos con *hyper-threading*, y parte del trabajo (reservar memoria para 10 000 notificaciones) compite por el recolector de basura, que es un recurso compartido. Es la ley de Amdahl, no un defecto de la implementación.

1.4.5. ¿Thread Pool o Reactive Streams?

La pregunta tiene trampa, porque las dos cosas resuelven problemas distintos y este sistema necesita los dos.

Lo que resuelve un pool de consumidores. Se midió vaciando una cola de notificaciones ya creadas, con dos clases de canal: uno instantáneo y otro que tarda 1 ms, como un proveedor real.

(a) Canal instantáneo, 200 000 avisos				**(b) Canal de 1 ms, 1 000 avisos**	
Cons.	**Encolar**	**Vaciar**	**Avisos/s**	**Vaciar**	**Avisos/s**
1	60 ms	105 ms	1 896 000	1 262 ms	792
2	119 ms	129 ms	1 550 000	577 ms	1 734
4	132 ms	138 ms	1 454 000	290 ms	3 444
8	97 ms	102 ms	1 965 000	156 ms	**6 409**

Tabla 22: El paralelismo paga cuando cada envío espera, y estorba cuando no.

Las dos mitades dicen lo mismo desde lados opuestos. Con un canal instantáneo, añadir consumidores solo añade contención sobre la cola (1 consumidor ya da 1,9 millones de avisos/s). Con un canal de 1 ms —lo normal en un proveedor real de email o SMS— la mejora es casi lineal: 8× más consumidores dan 8,1× más rendimiento.

Y como el consumidor espera con `await` y no bloquea un hilo del sistema, subir ese número es barato. Con los ∼800 avisos/s por consumidor que se midieron, atender los 10 000 avisos por tick del escenario B pediría del orden de **13 consumidores concurrentes** para despachar en un segundo.

Lo que un pool no resuelve: el backpressure. Si los cambios de precio llegan más rápido de lo que se puede notificar, ningún número de hilos arregla nada: la cola crece hasta agotar la memoria.

20 000 avisos de golpe, cola de 1 000, canal de 1 ms	
Encolar los 20 000 avisos	183,1 ms (el sujeto nunca esperó)
Aceptadas y entregadas	1 556
Rechazadas por cola llena	**18 444**
Tiempo total hasta vaciar	537 ms

Tabla 23: La cola acotada convierte la saturación en un número visible.

Perder 18 444 avisos es malo, pero **es mejor que la alternativa silenciosa**: con una cola sin límite, esos 18 444 se habrían acumulado, el proceso habría crecido sin freno y el fallo habría llegado más tarde, peor y sin explicación.

`BoundedChannelFullMode`	Qué hace	¿Sirve aquí?
`Wait`	el productor espera	**No** para `Encolar`: volvería a bloquear al hilo del precio (inciso c). Sí para los reintentos.
`DropOldest` / `DropNewest`	descarta un aviso	**No**: cada aviso es un evento distinto ("cruzó 152,00"), no la última foto de un estado. Descartar el más viejo pierde un cruce real.
Rechazar (`TryWrite` falla)	rechaza el nuevo y lo registra	**Sí**: es lo que hace `DespachadorAsincrono`, para que la saturación se vea.

Tabla 24: La política de saturación hay que elegirla explícitamente.

Y la respuesta de verdad a la saturación no es la política, sino **dimensionar**: más consumidores, particionar la cola por acción (cada acción ya es un Subject independiente) y, si aun así no se llega, degradar a propósito (agrupar en un resumen por cliente).

Por qué Channels y no Rx.NET. El enunciado propone "Reactive Streams". Conviene distinguir dos cosas que suelen confundirse:

- **Reactive Streams** es una *especificación* (`Publisher` / `Subscriber` / `Subscription`) cuyo rasgo central es precisamente el backpressure: el suscriptor pide con `request(n)` y el productor no puede enviar más de lo pedido. La demanda va **en el protocolo**.
- **Rx.NET** (`System.Reactive`, `IObservable<T>` / `IObserver<T>`) **no implementa esa especificación**. `OnNext` es *push* puro: no hay señal de demanda y el productor no tiene forma de saber que el consumidor va retrasado. Rx ofrece operadores para sobrevivir a la avalancha (`Sample`, `Throttle`, `Buffer`), pero todos **descartan o agrupan**; ninguno frena al productor.

Para este problema eso es decisivo, porque lo que hace falta es justo frenar o rechazar de forma controlada.

Pieza	Elección	Por qué
Detección	índice ordenado dentro de un ChangeManager	$O(\log n + k)$ en vez de $\Theta(n)$
Reparto de los k avisos	thread pool (`Parallel.For`) sobre el tramo	1,77× medido; ayuda cuando k es grande
Transporte	`Channel<T>` acotado del BCL	tiene el backpressure que a `IObservable<T>` le falta, sin dependencias nuevas
Envío	consumidores `async` dimensionados por la latencia del proveedor	6 409 avisos/s con 8 consumidores; escalan sin gastar hilos

Tabla 25: La propuesta, pieza a pieza.

`System.Threading.Channels` viene en la biblioteca base, no añade ningún paquete y ya expone `WriteAsync` (demanda) y `BoundedChannelFullMode` (política). Si en el futuro hiciera falta componer flujos (combinar acciones, ventanas temporales, *conflation* de avisos), Rx.NET sería un buen complemento **encima** de la cola, no en su lugar.

1.4.6. Propuesta final

1. **Indexar los umbrales por acción** dentro de un ChangeManager: de $\Theta(n)$ a $O(\log n + k)$. Es el único cambio que modifica el orden de complejidad, y es el que vale 731× con 1M de clientes.

2. **Mantener el índice en una estructura ordenada con inserción logarítmica** si las suscripciones cambian a menudo, en vez del array del prototipo.
3. **Particionar por acción**, que el diseño del inciso a ya permite: cada `Accion` es un Subject independiente con su propio índice y su propia cola.
4. **Cola acotada y consumidores `async`**, dimensionados con la latencia real del proveedor y con la política de saturación explícita y medida.
5. **Observar el sistema**: `Entregadas`, `Rechazadas`, `Reintentos`, `FallosDefinitivos` y el tamaño de la cola ya están expuestos como contadores.

1.4.7. Pruebas unitarias

Una optimización que cambia el comportamiento no es una optimización. `dotnet test` ejecuta **27 pruebas, todas correctas**; 7 son de este inciso.

Prueba	Qué comprueba
`NotificaExactamenteALosMismos-ClientesQueLasAlertasIndividuales`	**La prueba central.** 500 suscripciones con umbrales y direcciones aleatorias (semilla fija) y 40 cambios de precio: el índice y las alertas una por una producen exactamente el mismo conjunto de notificaciones, mismo cliente y mismo mensaje
`ElIndiceEnParaleloNotificaLo-MismoQueElSecuencial`	`Parallel.For` no cambia el resultado, solo el reparto
`NotificaSoloALosDelTramo-CruzadoAlAlza`	Quedan fuera los ya cruzados, los que están por encima del precio nuevo y los de dirección contraria
`NotificaSoloALosDelTramo-CruzadoALaBaja`	Lo mismo en el otro sentido
`RespetaLosExtremosDelTramo`	Umbral igual al precio anterior sí notifica; igual al precio nuevo, no. Es donde una búsqueda binaria se equivoca por uno
`NoNotificaCuandoNingunUmbral-QuedaEnElTramo`	Movimientos que no cruzan nada no generan avisos
`ElSujetoSoloVeUnObservador`	El índice es un `IObservadorPrecio`: la acción sigue sin conocer a los clientes

Tabla 26: Las pruebas del inciso d.

1.4.8. Ficheros de este inciso

Fichero	Qué es
`Rendimiento/GestorUmbralesIndexado.cs`	El ChangeManager con el índice ordenado y las búsquedas binarias
`Rendimiento/SuscripcionUmbral.cs`	La suscripción como dato (cliente, umbral, dirección)
`Rendimiento/DespachadorContador.cs`, `Rendimiento/RepositorioContador.cs`	Dobles que solo cuentan, para medir la detección sin el costo de la entrega
`Demos/BenchmarkRendimiento.cs`	El benchmark completo
`Observer/AlertaUmbral.cs`	**Modificado:** el texto del aviso se extrae a `Mensaje(...)` para que el índice genere exactamente el mismo mensaje

Tabla 27: Ficheros nuevos y modificados.

1.5. Restricción

El sistema debe garantizar entrega at-least-once *y persistir notificaciones fallidas.*

La entrega asíncrona del inciso c aísla los fallos y reintenta, pero toda esa estructura vive **en memoria**: si el proceso se cae, lo que estaba en vuelo se pierde. La restricción pide dos cosas que se apoyan en la misma decisión de diseño: **escribir la notificación en un almacén durable antes de intentar enviarla** (patrón *outbox*) y **no borrar nada que no se haya confirmado**. Con eso, un aviso que falló sigue en el disco y un aviso a medias se recupera al reiniciar.

La implementación es real y está en el repositorio: repositorio persistente en archivo, estados durables, recuperación al arrancar y siete pruebas que lo comprueban.

1.5.1. Respuesta corta

Pregunta	Respuesta
¿Cómo se garantiza *at-least-once*?	La notificación se escribe en el outbox **antes** de encolarse para el envío y solo se marca como `Entregada` cuando el canal confirma. Si el proceso se cae, lo pendiente sigue en el disco y otro proceso lo envía al arrancar.
¿Se puede enviar dos veces?	Sí: *at-least-once* lo permite. Por eso la fila del outbox se identifica por el `Id` de la notificación y el registro es un *upsert*: reenviar el mismo aviso no crea una segunda fila.
¿Qué pasa si el envío sigue fallando?	Se reintenta con espera creciente (*backoff* exponencial). Agotados los intentos, la notificación queda como `FalloDefinitivo` (*dead letter*) con su último error, y no se borra: un operador puede reintentarla.
¿Qué cambia respecto al inciso c?	Solo el almacén. El despachador sigue hablando con `IRepositorioNotificaciones` y, si el repositorio que recibe además implementa `IRepositorioOutbox`, persiste antes de enviar. Con un repositorio en memoria, el comportamiento es idéntico al del inciso c.

Tabla 28: La restricción en breve.

1.5.2. El outbox: escribir antes de enviar

El despacho del inciso c ya tenía la forma correcta: `Encolar` no espera a ningún canal y los consumidores envían en segundo plano. Lo que le faltaba era la **durabilidad**. La solución es una capacidad opcional del repositorio, `IRepositorioOutbox`, que el despachador detecta sin cambiar de contrato:

Listing 12: El despachador persiste antes de encolar (solo si el repositorio sabe hacerlo).

```
public void Encolar(Notificacion notificacion)
{
    // Outbox: la notificación queda en el disco ANTES de que se intente enviar.
    _outbox?.RegistrarPendiente(notificacion, intentos: 0);

    Interlocked.Increment(ref _enProceso);
    if (_cola.Writer.TryWrite(new EnvioPendiente(notificacion, Intentos: 0)))
    {
        return;
    }
    // ... cola llena: el aviso sigue pendiente en el outbox, no se pierde.
}
```

La línea que decide todo es la primera: entre `RegistrarPendiente` y `TryWrite` hay una ventana en la que el aviso **ya está en el disco y todavía no se ha enviado**. Si el proceso muere ahí, el aviso se recupera al siguiente arranque. Ese es el sentido de *at-least-once*: la entrega ocurre *al menos una vez*, y el precio de esa garantía es que puede ocurrir dos veces.

1.5.3. Los estados durables

Cada fila del outbox tiene tres estados, que son los que permiten saber qué se puede borrar y qué no:

Estado	Cuándo se pone	Qué implica
`Pendiente`	al encolar, antes de intentar el envío	se reintentará; si el proceso se cae, se recupera al reiniciar
`Entregada`	cuando el canal confirma	ya no hay nada que reintentar
`FalloDefinitivo`	al agotar los intentos (*dead letter*)	no se borra: queda con su último error para que un operador lo resuelva

Tabla 29: Los tres estados del outbox.

1.5.4. Idempotencia

Un aviso reenviado no puede crear una segunda fila. La clave es el `Id` de la `Notificacion`, que se genera al crearla y viaja con ella por todo el flujo:

Listing 13: El registro es un *upsert* por `Id`: reenviar no duplica.

```
private void Upsert(RegistroOutbox registro)
{
    if (!_registros.ContainsKey(registro.Id))
    {
        _orden.Add(registro.Id);
    }

    _registros[registro.Id] = registro;
    Volcar();
}
```

Y al registrar un aviso que ya estaba entregado, el repositorio no lo reabre:

```
public void RegistrarPendiente(Notificacion notificacion, int intentos)
{
    RegistroOutbox? existente = Buscar(notificacion.Id);

    // Idempotencia: un aviso ya entregado no se vuelve a abrir. Si el mismo
    // Id llega otra vez, es un reenvío, no una notificación nueva.
    if (existente is not null && existente.Estado == EstadoNotificacion.Entregada)
    {
        return;
    }
    // ...
}
```

1.5.5. Reintentos con backoff y *dead letter* persistido

El backoff del inciso c se conserva intacto; lo que se añade es que cada desenlace queda escrito:

Listing 14: Los tres desenlaces del envío, y lo que queda en el outbox.

```
// El canal confirma:
_repositorio.Guardar(envio.Notificacion);
```

```
_outbox?.MarcarEntregada(envio.Notificacion.Id);

// El canal falla y quedan intentos:
_outbox?.MarcarReintento(envio.Notificacion.Id, intentos, error.Message);
Reprogramar(envio with { Intentos = intentos });

// Se agotaron los intentos: dead letter, no se borra.
DarPorPerdida(envio.Notificacion, intentos, error.Message);
```

El contador de intentos **vive en el outbox**, no en la cola: por eso un reinicio no reinicia la cuenta. Un aviso que ya falló dos veces y se recupera al tercer intento sigue llevando `intentos = 2` después de la caída.

1.5.6. Reinicio y recuperación

Al arrancar, el proceso abre el outbox y devuelve a la cola lo que quedó pendiente:

Listing 15: Recuperar lo pendiente y reencolarlo con su contador de intentos.

```
public int RecuperarPendientes()
{
    IReadOnlyList<EnvioPendiente> pendientes = _outbox.PendientesParaReintentar();
    int recuperadas = 0;

    foreach (EnvioPendiente envio in pendientes)
    {
        if (_cola.Writer.TryWrite(envio))
        {
            Interlocked.Increment(ref _enProceso);
            recuperadas++;
        }
    }

    return recuperadas;
}
```

Un detalle que no es trivial: **el canal no se serializa**. Lo que se guarda en el archivo es el *tipo* del canal (por ejemplo `CanalEmail`), y al recuperar el aviso se reconstruye con una fábrica:

```
public sealed class FabricaCanales : IFabricaCanales
{
    public ICanalNotificacion? Crear(string tipoCanal) => tipoCanal switch
    {
        nameof(CanalEmail) => new CanalEmail(),
        nameof(CanalSms)   => new CanalSms(),
        nameof(CanalPush)  => new CanalPush(),
        _ => null,   // tipo desconocido: el aviso no se pierde, queda pendiente
    };
}
```

Si el tipo no se puede reconstruir, el aviso **no se descarta**: sigue pendiente en el outbox y queda listado en `NoRecuperables` para que un operador lo resuelva.

1.5.7. Dónde vive la persistencia

En este ejercicio, en un archivo JSON con reemplazo atómico: se escribe a un archivo temporal y se renombra, para que un corte a mitad de guardado no deje el outbox a medias.

Listing 16: La escritura al disco es atómica: temporal y renombrado.

```
private void Volcar()
{
    string temporal = _ruta + ".tmp";
```

```
    File.WriteAllText(temporal, json);
    File.Move(temporal, _ruta, overwrite: true);
}
```

Si el archivo está dañado, el repositorio **falla en voz alta** en vez de continuar como si el outbox estuviera vacío: un archivo corrupto significa notificaciones perdidas en silencio, y eso es justo lo que la restricción quiere evitar.

1.5.8. Límites del diseño

Conviene ser explícito sobre lo que esta solución *no* resuelve:

- **Un solo proceso escritor.** Dos instancias sobre el mismo archivo se pisan, porque cada una mantiene su propia copia en memoria. En producción esto se resuelve con un *outbox* transaccional en una base de datos, donde la escritura del aviso y la del evento de negocio van en la misma transacción.
- **No es exactly-once.** La garantía es *at-least-once*: el cliente puede recibir dos avisos si el proceso se cae entre la confirmación del canal y la escritura del estado. Para evitarlo harían falta claves de idempotencia del lado del proveedor.
- **El outbox no sustituye al proveedor.** Si la dirección de correo ya no existe, el aviso se reintenta, agota los intentos y queda en *dead letter*: la persistencia garantiza que la *notificación* no se pierde, no que el *cliente* la reciba.

1.5.9. Demostración

La demostración usa el código real (`dotnet run -project Bolsa.Notificaciones - restriccion`):

```
  1) Entrega normal: la alerta persiste el aviso y el canal lo confirma.
[EMAIL → cliente0001@correo.cu] AAPL subió por encima de su umbral 151.00: 150.00 → 155.00
  → Outbox: cliente0001@correo.cu → Entregada (intentos: 0)

  2) Canal caído: se reintenta con backoff y, al agotar los intentos, queda en dead letter.
[FALLO DespachadorAsincrono] intento 1/2 del aviso de +53 50000009 (InvalidOperationException: ...)
[FALLO DespachadorAsincrono] intento 2/2 del aviso de +53 50000009 (InvalidOperationException: ...)
  → Outbox: +53 50000009 → FalloDefinitivo (intentos: 2)
     último error: El proveedor rechazó el aviso de +53 50000009

  3) El proceso se cae con el aviso ya persistido y sin enviar.
  → Outbox antes de la caída: cliente0002@correo.cu → Pendiente (intentos: 0)
  [CAÍDA] el proceso se detiene.

  --- Reinicio: otro proceso abre el mismo archivo ---
  Pendientes leídos del disco: 1
  Avisos recuperados por el despachador: 1
[EMAIL → cliente0002@correo.cu] AAPL subió por encima de su umbral 151.00: 150.00 → 155.00
  → Outbox: cliente0002@correo.cu → Entregada (intentos: 0)

  4) Idempotencia: el mismo aviso encolado dos veces deja una sola fila en el outbox.
[EMAIL → cliente0003@correo.cu] AAPL 151.00 → 155.00
[EMAIL → cliente0003@correo.cu] AAPL 151.00 → 155.00
  → Filas del outbox: 3 antes, 4 después.
     Filas con ese mismo Id: 1 (el reenvío no duplica el registro).

  --- Resumen del outbox ---
     Entregadas: 3   Pendientes: 0   Fallo definitivo: 1
```

La escena 3 es la importante: el aviso se entrega *después* del reinicio, porque estaba en el disco antes de que el proceso muriera. Y la escena 4 muestra el otro lado de la moneda: el cliente puede recibir el aviso dos veces, pero el outbox guarda una sola fila.

1.5.10. Pruebas

`dotnet test` ejecuta **35 pruebas, todas correctas**; 7 son de esta restricción:

Prueba	Qué comprueba
`LaNotificacionQuedaPersistidaAntesDeEnviar`	Con el canal bloqueado dentro del envío, la fila del outbox está `Pendiente`; cuando el canal confirma, pasa a `Entregada`.
`ElPendienteSobreviveAlReinicio`	Otro proceso abre el mismo archivo y encuentra el aviso pendiente, con su mensaje, su contacto y su contador de intentos.
`LaNotificacionRecuperadaSeEntrega`	`RecuperarPendientes()` reencola el aviso, el canal lo recibe y la fila queda `Entregada`.
`ElFalloDefinitivoQuedaPersistido`	Canal siempre caído: la fila queda `FalloDefinitivo` con los 2 intentos y el último error.
`ElContadorDeIntentosSeConservaEnElDisco`	Canal que falla una vez y se recupera: la fila queda `Entregada` con `intentos = 1`.
`ElReenvioNoDuplicaLaFilaDelOutbox`	Registrar y guardar el mismo aviso varias veces deja una sola fila.
`UnOutboxDanadoNoSeDescartaEnSilencio`	Un archivo corrupto hace fallar la carga en vez de vaciar el outbox en silencio.

Tabla 30: Las pruebas de la restricción E9-R.

Las 28 pruebas de los incisos b, c y d siguen pasando: el cambio no altera su comportamiento.

1.5.11. Ficheros nuevos y modificados

Fichero	Qué es
`Entrega/EstadoNotificacion.cs`	**Nuevo.** Los tres estados durables: `Pendiente`, `Entregada`, `FalloDefinitivo`.
`Entrega/RegistroOutbox.cs`	**Nuevo.** La fila del outbox: la notificación serializable más el estado del envío.
`Entrega/IRepositorioOutbox.cs`	**Nuevo.** La capacidad opcional del repositorio: registrar pendiente, marcar entregada, reintento y fallo definitivo.
`Entrega/IFabricaCanales.cs`	**Nuevo.** Reconstruye el canal de un aviso al recuperarlo del disco.
`Entrega/RepositorioPersistente.cs`	**Nuevo.** El outbox en un archivo JSON, con escritura atómica y carga estricta.
`Entrega/DespachadorAsincrono.cs`	**Modificado.** Persiste antes de encolar, marca cada desenlace y añade `RecuperarPendientes()`.
`Demos/DemoPersistencia.cs`	**Nuevo.** La demostración de las cuatro escenas.
`Program.cs`	**Modificado.** Nuevo modo `restriccion`.
`Bolsa.Notificaciones.Tests/`	**Nuevo.** Las 7 pruebas de la restricción.

Tabla 31: Qué aporta la restricción E9-R al repositorio.

2. Ejercicio 12: Sistema de Procesamiento de Transacciones Bancarias

2.1. a) Patrón

Identifica el patrón de diseño más adecuado, justifica tu selección basándote en las características del problema y explica la propuesta de solución considerando las clases que intervienen y las relaciones entre ellas. ¿Por qué no usar Strategy o polimorfismo simple?

2.1.1. Patrón seleccionado: Visitor

Visitor representa una operación que se aplica a los elementos de una estructura de objetos, y permite definir operaciones nuevas sin cambiar las clases de los elementos sobre los que opera (Gamma et al., 1994, p. 366). La conferencia de la asignatura lo presenta como una forma de separar un algoritmo de la estructura de objetos sobre la que opera, siguiendo el principio abierto/cerrado (*Marcos de Trabajo y Patrones de Diseño*, diap. 40).

En este problema, los **elementos** son las cuentas, y las **operaciones** son visitantes: ejecutar, revertir y auditar, y las que vengan después, como «congelar cuenta» o «calcular intereses».

> El enunciado nombra tres tipos de cuenta (ahorro, corriente e inversión). El inciso b pide cuatro y agrega la cuenta empresarial, así que el diseño usa las cuatro.

2.1.2. Justificación

GoF recomienda Visitor en tres situaciones (Gamma et al., 1994, p. 368), y las tres se dan en este problema (las tres primeras filas de la tabla). Además, dos consecuencias del patrón resuelven otros dos requisitos del enunciado (las dos últimas filas):

Característica del problema	Cómo la resuelve Visitor
El comportamiento depende de dos cosas a la vez: de la operación y del tipo de cuenta. Un retiro no se ejecuta igual en una cuenta de ahorro (límite de retiros al mes), en una corriente (sobregiro) o en una de inversión (penalización por retiro anticipado).	**Doble despacho**: `cuenta.Aceptar(visitante)` elige el código según la operación y según la cuenta, sin `if` ni `switch` por tipo (ver más abajo). GoF lo recomienda cuando las operaciones dependen de la clase concreta de cada objeto (p. 368).
Las operaciones no tienen relación entre sí. Mover dinero, deshacerlo, auditar y calcular intereses son responsabilidades distintas.	Cada operación queda reunida en su propia clase y no se reparte por las cuentas (GoF, pp. 368, 371). Por ejemplo, toda la auditoría vive en `AuditarTransaccion`.
Los tipos de cuenta son estables y las operaciones crecen. Un banco crea productos nuevos muy de vez en cuando; en cambio, el enunciado ya anticipa operaciones nuevas.	Es el caso para el que GoF recomienda Visitor: las clases de la estructura rara vez cambian, pero a menudo se quieren operaciones nuevas (pp. 368, 371).
Hay que poder añadir operaciones («congelar cuenta», «calcular intereses») **sin modificar las cuentas ni las transacciones**.	Una operación nueva es **una clase visitante nueva**, y ninguna cuenta cambia. Según GoF, Visitor hace fácil añadir operaciones (p. 370). Es el principio abierto/cerrado aplicado a las operaciones.
La auditoría acumula resultados mientras recorre las cuentas: un registro por cuenta, y totales por sucursal en el inciso c.	Un visitante puede acumular estado mientras visita los elementos (GoF, p. 372).

Tabla 32: Características del problema y cómo las resuelve Visitor.

2.1.3. Propuesta de solución: clases y relaciones

El diseño se presenta en dos figuras. La primera muestra el patrón; la segunda, las transacciones y la auditoría. En la carpeta `diagramas` del repositorio hay una guía para leerlas.

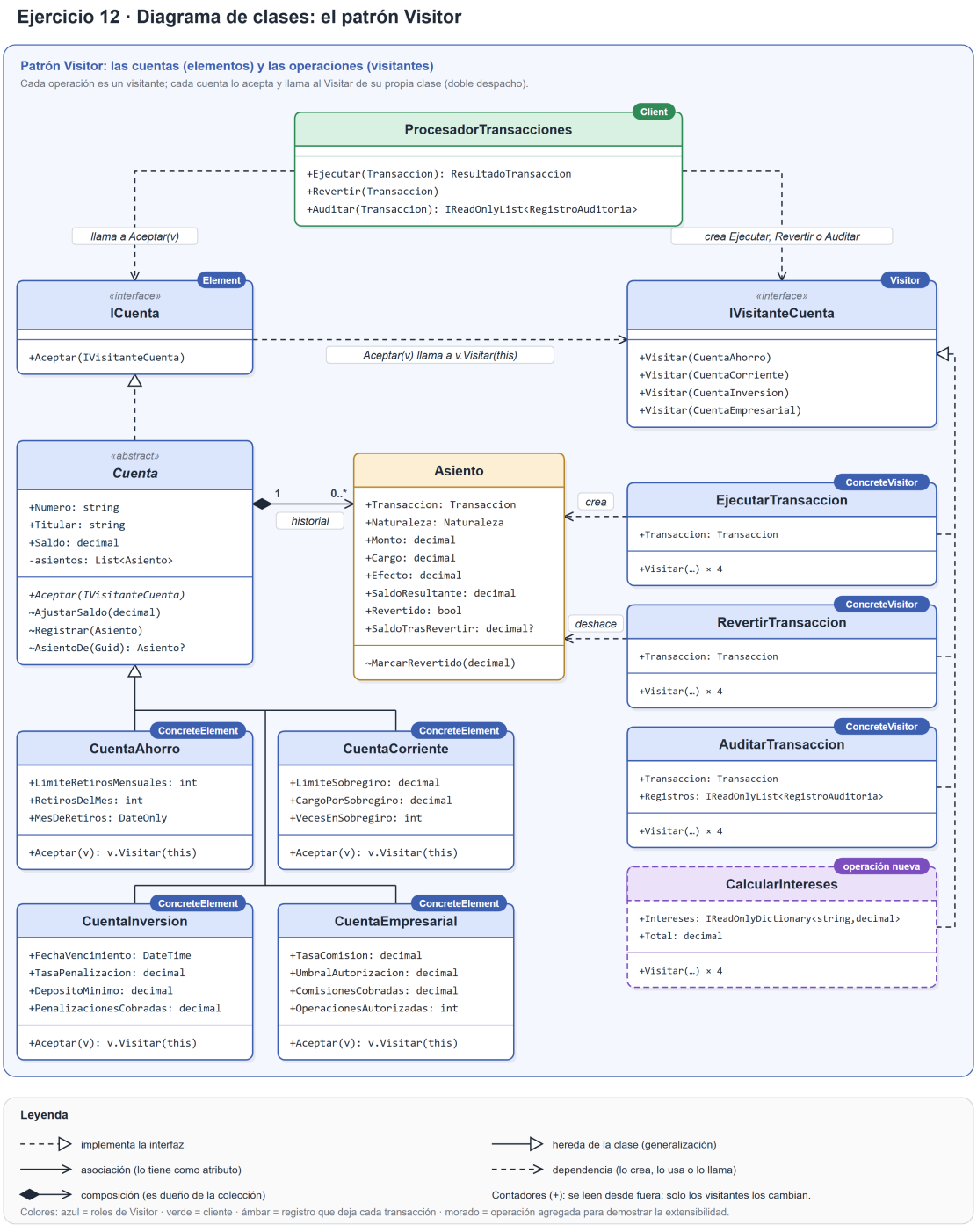


Figura 4: Diagrama de clases, figura 1: el patrón Visitor.

Ejercicio 12 · Diagrama de clases: transacciones, auditoría y «todo o nada»

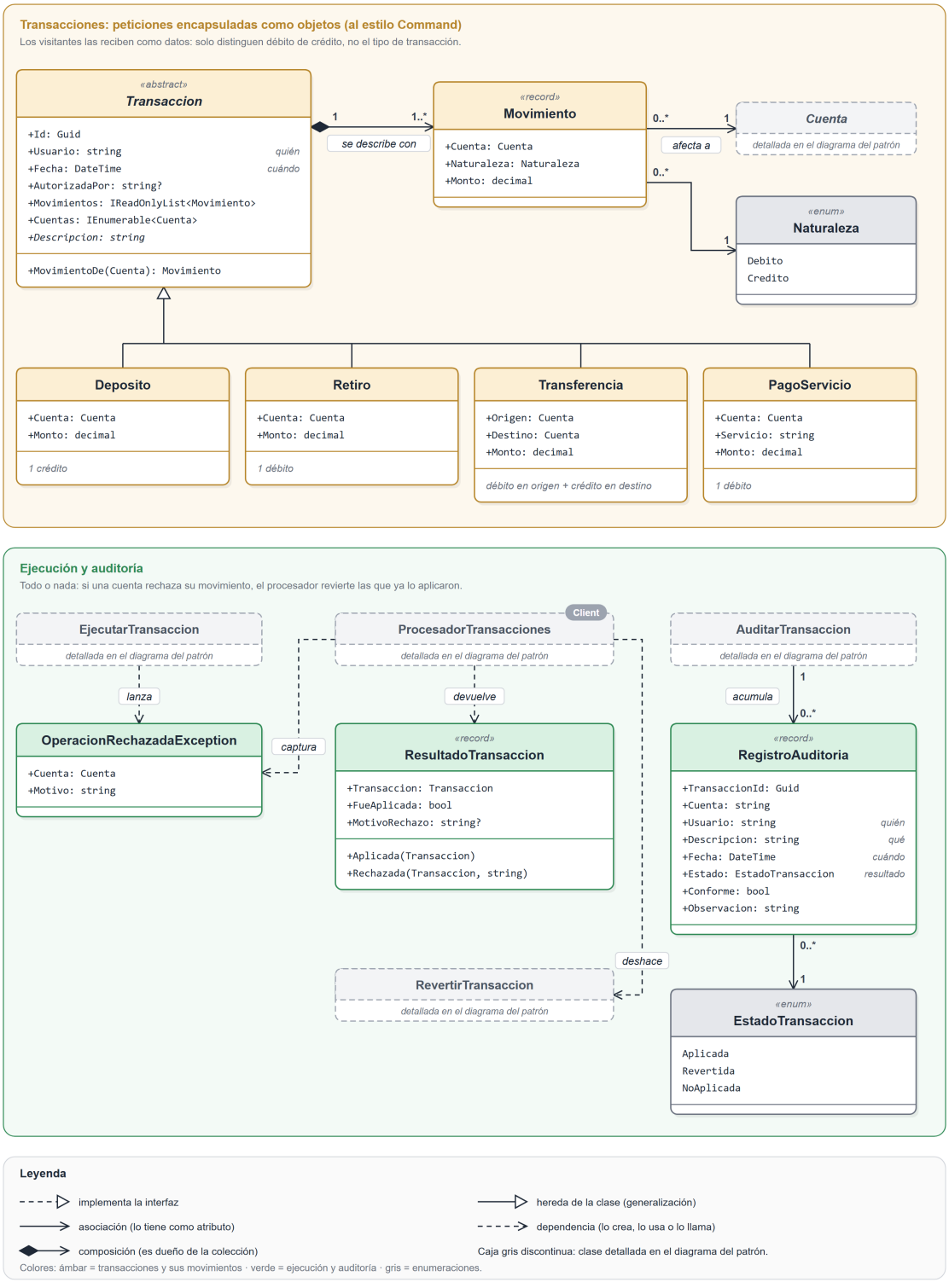


Figura 5: Diagrama de clases, figura 2: transacciones, auditoría y «todo o nada».

Rol (GoF)	Clase	Qué hace
Element	`ICuenta`	Declara `Aceptar(IVisitanteCuenta)` y nada más. El inciso c solo necesita esta interfaz para que un grupo de cuentas (Composite) se trate igual que una cuenta.
ConcreteElement	`CuentaAhorro`, `CuentaCorriente`, `CuentaInversion`, `CuentaEmpresarial`	Implementan `Aceptar` con una sola línea, `visitante.Visitar(this)`, y guardan los parámetros y contadores de su tipo. Heredan de la clase abstracta `Cuenta`, que tiene lo común: número, titular, saldo e historial de asientos.
Visitor	`IVisitanteCuenta`	Declara un `Visitar` por cada clase de cuenta (cuatro sobrecargas).
ConcreteVisitor	`EjecutarTransaccion`, `RevertirTransaccion`, `AuditarTransaccion`	Las tres operaciones del enunciado. Cada `Visitar` aplica la regla de esa operación para ese tipo de cuenta. `CalcularIntereses` es una cuarta operación, agregada en el inciso b para demostrar la extensibilidad.
ObjectStructure	las cuentas de una transacción	GoF admite que sea una colección o un Composite (p. 370). Aquí son las cuentas a las que afectan los movimientos de la transacción; en el inciso c será un grupo de cuentas.
Client	`ProcesadorTransacciones`	Crea el visitante de la operación (`EjecutarTransaccion`, `RevertirTransaccion` o `AuditarTransaccion`) y llama a `Aceptar` en cada cuenta de la transacción. Para pasar el visitante solo usa `ICuenta`: no sabe qué tipos de cuenta hay ni cuáles son sus reglas.

Tabla 33: Roles de Visitor en la solución.

Roles del patrón (figura 4).

Relaciones y multiplicidades. Las cuatro cuentas heredan de `Cuenta`, que implementa `ICuenta`. Los visitantes implementan `IVisitanteCuenta`. Cada `Cuenta` es dueña de 0 a muchos `Asiento` (composición). Cada visitante de transacción guarda la `Transaccion` que procesa (su atributo `Transaccion`). En la figura 5, cada `Transaccion` se describe con 1 o más `Movimiento`, y cada movimiento afecta a 1 `Cuenta`.

El doble despacho, sin `if` ni `switch`.

```
// En cada cuenta concreta (aquí, la corriente):
public override void Aceptar(IVisitanteCuenta visitante) => visitante.Visitar(this);

// En el cliente, que pasa el visitante a través de ICuenta:
cuenta.Aceptar(new EjecutarTransaccion(retiro));
```

1. **Primer despacho.** `cuenta.Aceptar(...)` es una llamada virtual: se ejecuta el `Aceptar` de la clase real de la cuenta, por ejemplo `CuentaCorriente`.

2. **Segundo despacho.** Dentro de ese `Aceptar`, `this` es de tipo `CuentaCorriente`, así que el compilador elige la sobrecarga `Visitar(CuentaCorriente)`. Como `visitante.Visitar` también es una llamada virtual, se ejecuta la de la operación real, `EjecutarTransaccion`.

El código que se ejecuta depende de dos tipos, el de la operación y el de la cuenta: es lo que GoF llama doble despacho (p. 374). En ningún momento se pregunta de qué tipo es la cuenta. Además, si se agrega un tipo de cuenta, el programa no compila hasta que todos los visitantes tienen su `Visitar`.

¿Dónde están las transacciones? Las transacciones (depósito, retiro, transferencia y pago de servicios) son **objetos que encapsulan una petición**, al estilo del patrón Command. GoF menciona que Command también se conoce como *Transaction* (p. 263) y que es una forma de modelar transacciones, entendidas como un conjunto de cambios sobre los datos (p. 266). La conferencia lo describe como encapsular una petición para poder encolarla, llevar un registro y deshacerla (diap. 34). Son justo las tres cosas que pide el enunciado: ejecutar, auditar y revertir.

En la figura 5:

- `Transaccion` (abstracta) guarda **quién** la pidió (`Usuario`), **cuándo** (`Fecha`), la autorización si hace falta y sus **movimientos**. Las cuatro transacciones del enunciado son subclases.
- Cada transacción se describe con **débitos y créditos** (`Movimiento`). Un depósito es un crédito; un retiro o un pago, un débito; una transferencia, un débito en el origen y un crédito en el destino.
- Cada movimiento apunta a una `Cuenta`, la clase base de las cuentas reales, y no a `ICuenta`: un depósito o un retiro afecta siempre a una cuenta concreta, nunca a un grupo de cuentas del inciso c. Un grupo se puede visitar, pero no puede ser el destino de un movimiento.
- Los visitantes **reciben la transacción como datos** en su constructor, por ejemplo `new EjecutarTransaccion(transaccion)`, y en cada cuenta aplican el movimiento que le corresponde.

Por qué no un Command completo, con `Ejecutar()` y `Deshacer()` dentro de cada transacción:

1. **Una operación nueva obligaría a modificar todas las transacciones.** «Auditar» o «calcular intereses» serían métodos nuevos en las cuatro clases de transacción, y el enunciado prohíbe modificarlas.
2. **Dentro de cada `Ejecutar()` habría que preguntar por el tipo de cuenta**, porque un retiro no se aplica igual en una cuenta de ahorro que en una corriente. Volverían los `if` y `switch` por tipo.

Con este reparto, los visitantes solo distinguen **débito de crédito**, no el tipo de transacción. Por eso un tipo de transacción nuevo, por ejemplo un pago de nómina (un débito y varios créditos), es solo otra subclase que describe sus movimientos: no cambia ningún visitante ni ninguna cuenta.

¿Qué se guarda para poder revertir? Cuando `EjecutarTransaccion` aplica un movimiento, deja en la cuenta un `Asiento` con la transacción, si fue débito o crédito, el monto y el **cargo** que cobró la regla de la cuenta (comisión, penalización o cargo por sobregiro). Es decir, se guarda **el efecto exacto que se aplicó**, no una foto del estado anterior. El asiento también anota el saldo en que la transacción dejó la cuenta, para que la auditoría la juzgue por lo que hizo.

`RevertirTransaccion` busca ese asiento y aplica la **operación inversa**: resta del saldo el efecto que tuvo (en un débito devuelve el monto y el cargo; en un crédito retira el monto). Después descuenta los contadores propios de cada tipo de cuenta: el retiro del mes en la de ahorro, el sobregiro en la corriente, la penalización en la de inversión, y la comisión y la autorización en la empresarial. Por último, marca el asiento como revertido para que no se deshaga dos veces.

Se descartó Memento, que guarda una foto del estado para restaurarla después (Gamma et al., 1994, p. 316; conferencia, diap. 38). Restaurar una foto **borra las transacciones que se aplicaron después** sobre la misma cuenta. Eso pasa siempre que se revierte una transacción que no es la última, y con operaciones en paralelo (la restricción del enunciado) es lo habitual. Por ejemplo:

Paso	**Saldo**
Saldo inicial	1000
Retiro de 100	900
Depósito de 50	950
Revertir el retiro restaurando la foto anterior	1000: **se pierde el depósito**
Revertir el retiro aplicando la operación inversa	**1050**, correcto

Tabla 34: Revertir con una foto del estado frente a revertir con la operación inversa.

GoF admite que un comando que se puede deshacer guarde los argumentos de la operación que aplicó (p. 269), y eso es lo que guarda el `Asiento`. También describe mementos que guardan solo el cambio incremental (p. 321), pero exigen deshacer en un orden predecible, y aquí no se cumple: se puede revertir cualquier transacción, no solo la última. La conferencia de Arquitecturas llama a esta técnica **compensación de transacciones** (diap. 58).

Decisiones de diseño.

- **Las reglas de negocio viven en los visitantes, no en las cuentas.** Las cuentas solo guardan sus datos y parámetros: límite de retiros, tope de sobregiro, tasas y umbral de autorización. Así cada operación nueva trae sus propias reglas sin tocar las cuentas.
- **Todo o nada.** Una transferencia afecta a dos cuentas. Si el débito ya se aplicó y el crédito se rechaza (por ejemplo, el destino es una cuenta de inversión y el monto no llega al depósito mínimo), `ProcesadorTransacciones` deshace el débito con el mismo `RevertirTransaccion`. Es la base de la atomicidad que pide la restricción.
- **Revertir es una compensación obligatoria.** No aplica las reglas de la cuenta: si se revierte un depósito que ya se gastó, el saldo puede quedar por debajo de lo que permite la cuenta, y `AuditarTransaccion` lo marca como no conforme. En un banco, una reversión (por ejemplo, la de un cheque sin fondos) no se puede rechazar por falta de saldo.
- **La auditoría juzga la transacción, no el estado actual de la cuenta.** Revisa lo que dice el asiento: el cargo cobrado, la autorización y el saldo en que la transacción (o su reversión) dejó la cuenta. Así, una transacción correcta sigue siendo conforme aunque después otras cambien la cuenta.

2.1.4. ¿Por qué no usar Strategy?

Strategy define una familia de algoritmos intercambiables para **un mismo contexto** (Gamma et al., 1994, p. 349; conferencia, diap. 39). El despacho es simple: el código que se ejecuta depende solo de la estrategia elegida. En este problema depende de dos cosas a la vez, y con Strategy una de las dos queda sin resolver:

- **Si cada operación fuera una estrategia** (`IOperacion.Aplicar(Cuenta)`), dentro de cada una habría que preguntar qué tipo de cuenta llegó:

```
1 if (cuenta is CuentaAhorro ahorro) { /* límite de retiros */ }
2 else if (cuenta is CuentaCorriente corriente) { /* sobregiro */ }
3 // ... y así con cada tipo de cuenta, en cada operación
```

Son justo los condicionales que Strategy debería eliminar (GoF, p. 352), y el antipatrón *Spaghetti Code* («muchos if o switch») de la conferencia (diap. 45). Además, si se agrega un tipo de cuenta, el compilador no avisa en qué estrategias falta su caso: el caso nuevo se ignora en silencio o falla en ejecución. Con Visitor, el programa no compila hasta que cada visitante tiene su `Visitar`.

- **Si cada cuenta tuviera su estrategia** (las reglas de ahorro, las de corriente...), cada operación nueva sería un método nuevo en la interfaz de la estrategia y en todas sus implementaciones. Es el mismo problema que el del polimorfismo simple (siguiente sección).
- **En Strategy, el cliente elige el algoritmo**, y por eso tiene que conocer las estrategias (GoF, p. 353). Aquí nadie elige: la regla la impone el tipo de cuenta.

Strategy sí sería adecuado para una variación de una sola dimensión, por ejemplo cambiar la fórmula de interés de un mismo tipo de cuenta.

2.1.5. ¿Por qué no usar polimorfismo simple?

Polimorfismo simple sería declarar un método virtual por operación en `Cuenta` (`Ejecutar`, `Revertir` y `Auditar`) y redefinirlo en cada cuenta.

- **Cada operación nueva obligaría a modificar las cuatro clases de cuenta**, además de la clase base: «calcular intereses» tocaría 5 clases y «congelar cuenta», otras 5. Es lo que el enunciado prohíbe («sin modificar las clases de las cuentas»). Con Visitor, cada operación es 1 clase nueva y 0 cuentas modificadas. El inciso d compara las dos opciones en detalle.
- **Las cuentas acumularían responsabilidades sin relación entre sí**: mover dinero, deshacerlo, auditar, calcular intereses... La clase de cada cuenta crecería con cada operación hasta convertirse en el antipatrón *The Blob* (clases gigantes) de la conferencia (diap. 45). GoF lo describe como «contaminar» las clases con operaciones (p. 368).

El polimorfismo simple solo conviene en el caso contrario: cuando los tipos de cuenta cambian a menudo y las operaciones son fijas (GoF, p. 371). No es lo que plantea el enunciado.

2.1.6. Desventajas de Visitor y cómo se acotan

- **Agregar un tipo de cuenta es caro.** Hay que agregar un `Visitar` a `IVisitanteCuenta` e implementarlo en todos los visitantes (GoF, p. 371): aquí, en los 3 del enunciado y en `CalcularIntereses`. Se acepta porque los tipos de cuenta son estables; el inciso d lo cuantifica.
- **Debilita la encapsulación.** Los visitantes tienen que cambiar el saldo y los contadores de la cuenta, así que la cuenta debe exponerlos (GoF, p. 372). Se acota con `internal`: los contadores tienen lectura pública, pero solo se pueden cambiar desde el ensamblado del banco, y fuera de él las cuentas solo se pueden leer.
- **Cada visitante tiene que implementar los cuatro `Visitar`**, aunque en una operación dos cuentas se traten igual. Se resuelve con métodos privados comunes dentro del visitante (por ejemplo, `Aplicar` en `EjecutarTransaccion`), sin copiar código.
- **Las sobrecargas de `Visitar` hacen menos evidente qué método se llama**, porque todas tienen el mismo nombre y solo cambia el tipo del parámetro. GoF lo presenta como una cuestión de preferencia (p. 380, nota 10); aquí se usan sobrecargas porque es lo habitual en C#.

2.1.7. Patrones relacionados

Patrón	Relación con esta solución
Command	Las transacciones son peticiones encapsuladas como objetos (ver arriba).
Memento	Se descartó para revertir: restaurar una foto del estado borraría transacciones posteriores.
Composite	En el inciso c, un grupo de cuentas implementa `ICuenta` y su `Aceptar` pasa el visitante a sus hijos. GoF lo señala como un uso típico de Visitor (p. 380).
Iterator	En el inciso c, recorre las transacciones de una cuenta sin exponer su lista interna.

Tabla 35: Patrones relacionados.

2.1.8. Usos conocidos

- **.NET:** `System.Linq.Expressions.ExpressionVisitor` recorre árboles de expresiones con un método `Visit...` por cada tipo de nodo. Los proveedores de LINQ lo usan; por ejemplo, Entity Framework lo usa para traducir consultas a SQL.
- **La API de Roslyn (el compilador de C#):** `CSharpSyntaxVisitor` tiene un `Visit...` por cada tipo de nodo del árbol de sintaxis. Es el mismo ejemplo con el que GoF presenta el patrón: un compilador que aplica muchas operaciones a un árbol de sintaxis (p. 366).

Referencias. Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. Los números de página corresponden a la copia del libro en la bibliografía de la asignatura (*Documentacion Patrones*), igual que en el ejercicio 9.

Ingeniería de Software. *Marcos de Trabajo y Patrones de Diseño* [diapositivas de la conferencia].

Ingeniería de Software. *Arquitecturas de Software* [diapositivas de la conferencia].

2.2. b) Implementación

> *Implementa con 4 tipos de elementos (CuentaAhorro, CuentaCorriente, CuentaInversión, CuentaEmpresarial) y 3 operaciones (EjecutarTransacción, RevertirTransacción, AuditarTransacción). Cada operación debe comportarse de manera diferente según el tipo de cuenta.*

La implementación sigue el diseño del inciso a: tiene las mismas clases y relaciones que sus dos diagramas de clases (figuras 4 y 5). El código completo está en la carpeta `codigo` del ejercicio, en el repositorio; aquí se muestran solo los fragmentos relevantes.

2.2.1. Cómo ejecutarlo

Requisito: SDK de .NET 10. Desde `ejercicio-12-transacciones-bancarias/codigo/`:

```
dotnet run --project Banco.Procesamiento   # la simulación
dotnet test                                 # las pruebas unitarias (xUnit)
```

Hay que indicar `--project` porque la carpeta tiene dos proyectos (el programa y sus pruebas). Las fechas de la simulación son fijas, así que la salida es la misma en cada ejecución.

2.2.2. Estructura del código

Carpeta / fichero	Clase del diagrama y rol
`Cuentas/ICuenta.cs`	`ICuenta`: Element
`Cuentas/Cuenta.cs`	`Cuenta`: base común de las cuentas (número, titular, saldo e historial)
`Cuentas/Cuenta*.cs`	Las 4 cuentas: ConcreteElement
`Cuentas/Asiento.cs`	`Asiento`: lo que una transacción le hizo a una cuenta; lo que se guarda para revertir y auditar
`Visitantes/IVisitanteCuenta.cs`	`IVisitanteCuenta`: Visitor
`Visitantes/EjecutarTransaccion.cs`, `RevertirTransaccion.cs`, `AuditarTransaccion.cs`	Las 3 operaciones: ConcreteVisitor
`Visitantes/CalcularIntereses.cs`	ConcreteVisitor nuevo: demuestra la extensibilidad
`Visitantes/RegistroAuditoria.cs`, `EstadoTransaccion.cs`, `OperacionRechazadaException.cs`	Resultados de auditar y de rechazar
`Transacciones/Transaccion.cs` y sus 4 subclases	`Transaccion`, `Deposito`, `Retiro`, `Transferencia`, `PagoServicio`: peticiones encapsuladas como objetos (al estilo Command)
`Transacciones/Movimiento.cs`, `Naturaleza.cs`	Los débitos y créditos de cada transacción
`ProcesadorTransacciones.cs`, `ResultadoTransaccion.cs`	Client: crea los visitantes y garantiza «todo o nada»
`Dinero.cs`	Clase auxiliar interna: redondeo y formato de importes (no aparece en los diagramas)
`Program.cs`	La simulación
`Banco.Procesamiento.Tests/`	Pruebas unitarias

Tabla 36: Ficheros del proyecto y clase del diagrama que implementa cada uno.

2.2.3. Cada operación se comporta distinto según la cuenta

Las reglas de negocio son inventadas, pero explícitas: cada una está documentada en el `Visitar` que la aplica, y sus valores son parámetros del constructor de la cuenta.

Cuenta	`EjecutarTransaccion`	`RevertirTransaccion`	`AuditarTransaccion`
Ahorro	Admite 3 débitos por mes (retiros, transferencias salientes y pagos), según la fecha de la transacción: el cuarto del mismo mes se rechaza y un mes nuevo empieza de cero. Sin sobregiro: nunca deja el saldo en negativo.	Devuelve el monto y, si era un débito del mes en curso, descuenta 1 de `RetirosDelMes`.	Conforme si la transacción (o su reversión) no dejó el saldo en negativo.
Corriente	Permite quedar en negativo hasta -500. Cada débito que deja el saldo en negativo cobra un cargo fijo de 5 y suma 1 a `VecesEnSobregiro`; el tope se comprueba con el cargo incluido.	Devuelve el monto y el cargo y, si el débito había dejado la cuenta en negativo, descuenta 1 de `VecesEnSobregiro`.	Comprueba que el cargo se cobrara solo si el débito dejó la cuenta en negativo, y que el saldo no bajara del tope.
Inversión	Los créditos tienen un mínimo de 100. Un débito antes de la fecha de vencimiento paga una penalización del 2 %.	Devuelve el monto y la penalización y la descuenta de `PenalizacionesCobradas`.	Comprueba la penalización según la fecha, el depósito mínimo y que el saldo no quedara negativo.
Empresarial	Cada débito paga una comisión del 0,5 %. Los débitos de más de 10 000 necesitan la autorización de alguien distinto de quien pide la transacción.	Devuelve el monto y la comisión y, si era un monto alto, descuenta 1 de `OperacionesAutorizadas`.	Comprueba la comisión, que los montos altos vengan autorizados por otra persona y que el saldo no quedara negativo; anota quién autorizó.

Tabla 37: Reglas de cada operación según el tipo de cuenta.

En las cuatro cuentas se cumple además lo siguiente:

- **Revertir restaura exactamente el estado anterior**, saldo y contadores, porque aplica la operación inversa del `Asiento` que dejó la ejecución (inciso a). La única excepción es deliberada: si se revierte un débito de ahorro de un mes que ya terminó, el contador del mes en curso no cambia.
- **Simplificación de la cuenta de ahorro:** solo lleva el conteo del mes más reciente, así que se supone que las transacciones llegan en orden de fecha.
- **Auditar no modifica la cuenta.** Por cada cuenta genera un `RegistroAuditoria` con quién pidió la transacción (`Usuario`), qué se pidió (`Descripcion`), cuándo (`Fecha`) y el resultado (`Estado`: aplicada, revertida o no aplicada), además de si lo que la transacción le hizo a la cuenta cumple las reglas de su tipo.
- **El estado es por cuenta, no por transacción.** `Revertida` incluye la compensación automática del «todo o nada»: en una transferencia rechazada, la cuenta que alcanzó a aplicar el débito queda como `Revertida` (con el rastro de la anulación, como un asiento contable de anulación) y la otra como `NoAplicada`.
- **La auditoría juzga la transacción, no el estado actual de la cuenta.** Revisa lo que dice el asiento: el cargo cobrado, la autorización y el saldo en que la transacción (o su reversión) dejó la cuenta. Una transacción correcta sigue siendo conforme aunque después otras cambien la cuenta.

Listing 17: `EjecutarTransaccion` en una cuenta corriente.

```
public void Visitar(CuentaCorriente cuenta)
{
    Movimiento movimiento = MovimientoPendiente(cuenta);
    if (movimiento.Naturaleza == Naturaleza.Credito)
    {
        Aplicar(cuenta, movimiento, cargo: 0m);
        return;
    }

    bool quedaEnNegativo = cuenta.Saldo - movimiento.Monto < 0m;
    decimal cargo = quedaEnNegativo ? cuenta.CargoPorSobregiro : 0m;
    ExigirSaldo(cuenta, movimiento.Monto + cargo, saldoMinimo: -cuenta.LimiteSobregiro,
                $"supera el sobregiro permitido de {Dinero.Formato(cuenta.LimiteSobregiro)}");
    Aplicar(cuenta, movimiento, cargo);
    if (quedaEnNegativo)
    {
        cuenta.VecesEnSobregiro++;
    }
}
```

El método recibe una `CuentaCorriente` y no una cuenta genérica: lo eligió el doble despacho. Por eso puede leer `LimiteSobregiro` y `CargoPorSobregiro` sin preguntar por el tipo ni hacer un cast. Ningún visitante usa `is`, `switch` ni casts por tipo de cuenta. Se puede comprobar con la siguiente orden, que excluye los comentarios de documentación (`///`): están en español, y algunas versiones de `grep` encuentran «as» dentro de «así».

```
grep -rnE "\b(is|switch|as)\b" Banco.Procesamiento/Visitantes/ --include='*.cs' |
  grep -v "///"   # no devuelve nada
```

Ningún `Visitar` es una copia de otro: cada uno aplica la regla de su cuenta. Las partes comunes (aplicar el asiento y exigir saldo) son métodos privados del visitante (`Aplicar`, `ExigirSaldo`).

2.2.4. «Todo o nada» en una transacción

Una transferencia afecta a dos cuentas. `ProcesadorTransacciones` le pasa el visitante a cada una y, si una rechaza su movimiento, revierte las que ya lo aplicaron, en orden inverso, con `RevertirTransaccion`:

Listing 18: `ProcesadorTransacciones.Ejecutar`: todo o nada.

```
public ResultadoTransaccion Ejecutar(Transaccion transaccion)
{
    EjecutarTransaccion ejecutar = new(transaccion);
    List<ICuenta> aplicadas = [];
    try
    {
        foreach (ICuenta cuenta in transaccion.Cuentas)
        {
            cuenta.Aceptar(ejecutar);
            aplicadas.Add(cuenta);
        }

        return ResultadoTransaccion.Aplicada(transaccion);
    }
    catch (OperacionRechazadaException rechazo)
    {
        Deshacer(transaccion, aplicadas);
        return ResultadoTransaccion.Rechazada(transaccion, rechazo.Message);
    }
    catch
    {
```

```
        Deshacer(transaccion, aplicadas);
        throw;
    }
}
```

- Las reglas comprueban todo **antes** de cambiar la cuenta: una cuenta que rechaza su movimiento queda intacta, y solo hay que deshacer las anteriores.
- Si la transacción falla por otra causa (por ejemplo, porque ya se había ejecutado en una de sus cuentas), también se deshace lo aplicado, y después se propaga el error.
- Los constructores de las transacciones garantizan que haya como mucho un movimiento por cuenta, y que los montos no tengan fracciones de centavo.
- La restricción del enunciado (issue #10) parte de este mecanismo para garantizar la atomicidad cuando las transacciones corren en paralelo.

2.2.5. Extensibilidad: una operación nueva sin tocar las cuentas

`CalcularIntereses` es una cuarta operación. Para agregarla **solo se creó esa clase**, en la carpeta `Visitantes`: no se modificó ninguna cuenta, ninguna transacción ni ninguno de los otros visitantes. También se comporta distinto según la cuenta:

Cuenta	Interés del mes
Ahorro	3 % anual sobre el saldo positivo
Corriente	No paga intereses; si está en sobregiro, cobra el 18 % anual sobre lo que debe
Inversión	6 % anual sobre el saldo positivo
Empresarial	1 % anual, solo sobre lo que pase de 20 000

Tabla 38: Regla de `CalcularIntereses` según la cuenta.

```
CalcularIntereses intereses = new();
foreach (Cuenta cuenta in cuentas)
{
    cuenta.Aceptar(intereses);   // el mismo Aceptar de siempre: las cuentas no cambiaron
}
```

2.2.6. Resultados de la simulación

`Program.cs` crea una cuenta de cada tipo y ejecuta 12 transacciones de los 4 tipos. Algunas están pensadas para que se rechacen, una por cada regla:

```
--- 1. EjecutarTransaccion ---
  Aplicada   Depósito de 250.00 en AHO-001
  Aplicada   Retiro de 100.00 de AHO-001
  Aplicada   Retiro de 50.00 de AHO-001
  Aplicada   Pago de Electricidad por 80.00 desde AHO-001
  Rechazada  Retiro de 20.00 de AHO-001
             motivo: AHO-001: alcanzó el límite de 3 retiros del mes
  Aplicada   Retiro de 600.00 de COR-001
  Rechazada  Retiro de 300.00 de COR-001
             motivo: COR-001: supera el sobregiro permitido de 500.00 (saldo -305.00, a debitar 305.00)
  Aplicada   Retiro de 1000.00 de INV-001
  Rechazada  Transferencia de 50.00 de COR-001 a INV-001
             motivo: INV-001: el depósito mínimo es 100.00
  Rechazada  Transferencia de 15000.00 de EMP-001 a INV-001
             motivo: EMP-001: los débitos de más de 10000.00 necesitan la autorización de otra persona
  Aplicada   Transferencia de 15000.00 de EMP-001 a INV-001
  Aplicada   Pago de Telefonía por 1200.00 desde EMP-001
```

- **8 aplicadas y 4 rechazadas.** Cada rechazo corresponde a una regla distinta: el límite de retiros, el tope de sobregiro, el depósito mínimo y la autorización.
- **Todo o nada.** La transferencia de 50 de COR-001 a INV-001 llegó a aplicar el débito en COR-001, con su cargo por sobregiro. Cuando INV-001 rechazó el crédito, el procesador revirtió ese débito, y COR-001 siguió en −305, con 1 vez en sobregiro.
- **Revertir** el retiro de 600 deja COR-001 en 300 y con 0 veces en sobregiro: el mismo estado que antes del retiro.
- **Auditar** genera 15 registros, uno por transacción y cuenta. Las cuentas a las que no llegó una transacción aparecen como `NoAplicada`. COR-001 aparece como `Revertida` en el retiro de 600 y en la transferencia rechazada, porque en las dos aplicó el débito y después se deshizo. Cada registro indica el saldo en que la transacción, o su reversión, dejó la cuenta.
- **Intereses del mes:** 2.55 (ahorro), 0.00 (corriente, ya sin sobregiro), 169.90 (inversión) y 11.43 (empresarial).

2.2.7. Pruebas

`dotnet test` ejecuta **46 pruebas xUnit**, todas correctas. Para cada combinación de cuenta y operación (4×3) hay al menos una prueba:

Cuenta	Ejecutar	Ejecutar y luego revertir	Auditar
Ahorro	retiro cuenta como débito del mes; depósito no; rechaza el cuarto débito del mes; un mes nuevo empieza de cero; rechaza sin saldo; permite dejar el saldo en cero	un retiro y un depósito dejan el estado original; revertir un retiro de un mes anterior no cambia el mes en curso	registra quién, qué, cuándo y resultado sin modificar la cuenta
Corriente	sin cargo si no queda en negativo; cargo al quedar en negativo; llega justo al tope; rechaza si pasa del tope	devuelve monto y cargo y descuenta el sobregiro	informa el sobregiro y el cargo sin modificar la cuenta; juzga la transacción y no el estado actual
Inversión	penalización antes del vencimiento; sin penalización desde el vencimiento; rechaza depósito menor que el mínimo; rechaza si la penalización deja sin saldo	devuelve monto y penalización	señala el retiro anticipado y su penalización; marca no conforme la reversión que deja la cuenta en negativo
Empresarial	cobra comisión en débitos; no en créditos; rechaza monto alto sin autorización; rechaza que alguien se autorice a sí mismo; acepta monto alto autorizado; un monto igual al umbral no necesita autorización	devuelve monto y comisión y descuenta la autorización	registra la comisión y quién autorizó sin modificar la cuenta

Tabla 39: Pruebas de cada combinación de cuenta y operación.

Cada prueba de «ejecutar y luego revertir» compara el saldo **y** los contadores de la cuenta con los de antes de ejecutar. Además:

- `ProcesadorTransaccionesTests` (7 pruebas):
  - una transferencia aplica el débito y el crédito;

- si el crédito se rechaza, se revierte el débito (todo o nada);
- si la transacción falla por otra causa, se deshace lo aplicado y se propaga el error;
- revertir no borra lo que otra transacción hizo después en la misma cuenta;
- revertir o ejecutar dos veces la misma transacción falla sin tocar la cuenta;
- un depósito ya gastado se revierte igual, y la auditoría lo marca como no conforme.

- `TransaccionTests` (3 pruebas): como mucho un movimiento por cuenta, ninguna transferencia a la misma cuenta y ningún monto con fracciones de centavo.
- `DobleDespachoTests`: a través de `ICuenta`, `Aceptar` llama al `Visitar` de la clase concreta.
- `CalcularInteresesTests` (3 pruebas): la operación nueva aplica la regla de cada cuenta, no da interés sobre un saldo negativo y no modifica nada.

2.3. c) Integración

Integra con Composite para manejar grupos de cuentas (ej. «Cuentas de una sucursal») y con Iterator para recorrer las transacciones de una cuenta. ¿Cómo afecta el Composite a la ejecución de operaciones sobre grupos?

Se integra sobre el código del inciso b. El grupo de cuentas entra por la interfaz `ICuenta` del inciso a, que solo declara `Aceptar`.

2.3.1. Composite: grupos de cuentas

Composite compone objetos en estructuras de árbol y permite que el cliente trate igual a un objeto individual y a una composición de objetos (Gamma et al., 1994, p. 183). Aquí el objeto individual es una cuenta y la composición es un grupo: una sucursal, una región o el banco entero.

Rol (GoF)	Clase	Qué hace
Component	`ICuenta`	Lo que tienen en común una cuenta y un grupo: `Aceptar(IVisitanteCuenta)`.
Leaf	las 4 cuentas (a través de `Cuenta`)	Llaman al `Visitar` de su clase.
Composite	`GrupoCuentas`	Guarda sus hijos (cuentas u otros grupos) y en su `Aceptar` le pasa el visitante a cada uno.
Client	`ProcesadorTransacciones`, `Program.cs`	Le pasan un visitante a una cuenta o a un grupo sin distinguirlos.

Tabla 40: Roles de Composite en la solución.

Listing 19: `GrupoCuentas.Aceptar`: el recorrido recursivo del árbol.

```
public void Aceptar(IVisitanteCuenta visitante)
{
    visitante.EntrarEnGrupo(this);
    foreach (ICuenta hijo in _hijos)
    {
        hijo.Aceptar(visitante);   // si el hijo es un grupo, recorre a su vez los suyos
    }

    visitante.SalirDeGrupo(this);
}
```

- **El cliente trata igual a una cuenta y a un grupo.** `cuenta.Aceptar(v)` y `banco.Aceptar(v)` se escriben igual, y `EjecutarEnCadaCuenta(ICuenta destino, ...)` acepta cualquiera de los dos. GoF lo presenta como la ventaja principal del patrón: el cliente no sabe, ni le importa, si tiene una hoja o un compuesto (p. 186).
- **Los grupos se anidan** (sucursal → región → banco), y el recorrido es **recursivo**: el `Aceptar` de un grupo llama al `Aceptar` de cada hijo. `Agregar` rechaza los ciclos (un grupo dentro de sí mismo o de uno de sus descendientes), porque el recorrido no terminaría.
- **Es el uso de Visitor que GoF señala** (p. 380): el elemento compuesto recorre sus hijos en su `Accept` (p. 373), y así el recorrido del árbol es responsabilidad de la estructura, no del visitante (p. 374).

Cuánto costó agregar el Composite. `IVisitanteCuenta` ganó dos métodos, `EntrarEnGrupo` y `SalirDeGrupo`, con una **implementación vacía por omisión**. Los visitantes que no distinguen grupos (ejecutar, revertir, auditar y calcular intereses) heredan esa implementación y **no cambiaron**. GoF admite esta implementación por omisión como excepción (p. 371). El costo total fue:

- 1 clase nueva (`GrupoCuentas`);
- 1 interfaz con 2 métodos nuevos;
- 0 cuentas y 0 visitantes modificados.

El inciso d usa estos números.

2.3.2. Iterator: las transacciones de una cuenta

Iterator permite recorrer los elementos de un agregado **sin exponer su representación interna** (Gamma et al., 1994, p. 289), y admitir varios recorridos distintos del mismo agregado (p. 291). En C# es `IEnumerable<T>`, como en las colecciones de .NET que muestra la conferencia (diap. 35–37).

Rol (GoF)	Aquí
Aggregate	`Cuenta`, que guarda su historial de asientos en una lista privada
CreateIterator	`Historial()`, y también `GrupoCuentas.Hijos` para los hijos de un grupo
Iterator	el `IEnumerator<Asiento>` que genera el compilador a partir de `yield return`

Tabla 41: Roles de Iterator en la solución.

```
public IEnumerable<Asiento> Historial()
{
    foreach (Asiento asiento in _asientos)
    {
        yield return asiento;
    }
}
```

- **No se expone la lista interna.** `Historial()` no devuelve la `List<Asiento>`: convertirla a `ICollection<Asiento>` para agregar o quitar asientos da `null` (hay una prueba).
- **Hay iteradores filtrados:** `Historial(desde, hasta)` filtra por fecha, y `Historial<Transferencia>()` por tipo de transacción.

- **Incluye los asientos revertidos**, marcados como tales: el historial es el rastro completo de la cuenta.
- **Si la cuenta cambia mientras se recorre, el recorrido falla** en vez de devolver datos a medias. Esto importa para la restricción (#10), donde varias transacciones corren en paralelo.
- **Interno y externo.** `Historial()` es un iterador *externo*: el cliente controla el bucle. El recorrido del árbol de grupos, en cambio, es *interno*: lo hace `Aceptar`. GoF explica que los iteradores externos son difíciles de implementar sobre estructuras recursivas como las de Composite, porque la posición abarca varios niveles (p. 295), y por eso el árbol lo recorre el propio Composite.

2.3.3. ¿Cómo afecta el Composite a la ejecución de operaciones sobre grupos?

1. La operación llega a todas las hojas, en profundidad y en el orden de inserción. Una operación sobre un grupo se aplica a **todas las cuentas del árbol**, no a los grupos: los grupos solo reenvían el visitante. El orden es un recorrido en profundidad que respeta el orden en que se agregaron los hijos. Con el árbol de la simulación:

```
Banco
+- Region Occidente
|  +- Sucursal Centro:  AHO-001, COR-001
|  +- Sucursal Playa:   INV-001, AHO-002
+- Sucursal Empresas:   EMP-001
```

el orden es AHO-001, COR-001, INV-001, AHO-002 y EMP-001. El visitante recibe además `EntrarEnGrupo` y `SalirDeGrupo` alrededor de cada grupo; la prueba `Aceptar_RecorreLosGruposAnidadosEnProfundidadYEnOrden` fija esa secuencia completa.

2. Si falla en una cuenta del grupo: mejor esfuerzo con reporte. `ProcesadorTransacciones.EjecutarEnCadaCuenta(destino, crear)` aplica una operación a una cuenta o a un grupo. Funciona así:

1. Obtiene las cuentas del destino con `RecolectarCuentas`, un visitante que recorre el árbol.
2. Crea **una transacción por cuenta** y la ejecuta con el «todo o nada» del inciso b.
3. Si una cuenta rechaza la suya, **las demás siguen**, y el resultado de cada una (aplicada, o rechazada con su motivo) queda en el reporte.

Por qué mejor esfuerzo y no todo o nada para el grupo:

- **Cada cuenta es de un cliente distinto.** Un cobro de mantenimiento a una sucursal son N hechos independientes, no uno solo. Si un cliente no tiene fondos, no hay razón para dejar de cobrarles a los demás.
- **Con todo o nada, una sola cuenta bloquearía al banco entero.** En la simulación, 2 de 5 cuentas rechazan el cobro (una agotó sus débitos del mes y otra no tiene saldo). Con todo o nada no se habría cobrado a ninguna, y con miles de cuentas casi siempre habrá alguna que rechace.
- **No se pierde ni se oculta nada.** Cada transacción por cuenta sigue siendo atómica, una cuenta que rechaza queda intacta, y el reporte dice qué cuentas fallaron y por qué, para reintentarlas o avisar al cliente.

Cuándo sí conviene todo o nada: cuando la operación es **un solo hecho** que toca varias cuentas, como una transferencia o una nómina (un débito en la empresa y un crédito a cada empleado). Ahí no tiene sentido aplicar una parte. Eso se modela como **una** transacción con varios movimientos, y `ProcesadorTransacciones.Ejecutar` ya la aplica con todo o nada (inciso b).

	Mejor esfuerzo (`EjecutarEnCadaCuenta`)	**Todo o nada (`Ejecutar`)**
Qué es	N transacciones independientes, una por cuenta	1 transacción con N movimientos
Si una cuenta rechaza	las demás se aplican; el reporte la señala	se revierten las ya aplicadas
Ejemplos	cobro de mantenimiento, abono de intereses	transferencia, pago de nómina

Tabla 42: Las dos políticas ante un fallo en una cuenta.

Por qué no basta con `grupo.Aceptar(new EjecutarTransaccion(t))`: `EjecutarTransaccion` aplica los movimientos de *una* transacción, que no tiene movimientos en las demás cuentas del grupo. Además, el procesador necesita el resultado de cada cuenta, para reportarlo o para revertirlo. Por eso primero obtiene las hojas con un visitante y después trabaja cuenta por cuenta.

3. Cómo un visitante acumula resultados al recorrer un grupo. `ResumirPorGrupo` calcula en **un solo recorrido** los totales de cada grupo: cuántas cuentas tiene, su saldo y los cargos cobrados.

- En `EntrarEnGrupo` abre un acumulador para el grupo, y en `SalirDeGrupo` lo cierra.
- Cada cuenta visitada **suma en todos los grupos abiertos**, es decir, en todos los que la contienen: una cuenta de la Sucursal Centro cuenta en su sucursal, en la Región Occidente y en el Banco.
- **Cada tipo de cuenta aporta sus cargos a su manera:** ahorro no cobra; corriente aporta el cargo por cada vez que quedó en sobregiro; inversión, sus penalizaciones; empresarial, sus comisiones. Es el doble despacho otra vez.

Es la consecuencia de Visitor que GoF llama acumular estado durante el recorrido (p. 372). Un visitante que no distingue grupos también acumula: `CalcularIntereses`, del inciso b, calcula los intereses de todas las cuentas del banco con `banco.Aceptar(...)`, **sin ningún cambio**.

2.3.4. Resultados de la simulación

`Program.cs` agrega las secciones 5 y 6 al final de la simulación del inciso b, con una cuenta más (AHO-002, con 3.00 de saldo):

```
--- 5. Composite: totales por grupo en un solo recorrido (ResumirPorGrupo) ---
  Banco                 5 cuentas  saldo  69022.00  cargos cobrados 101.00
    Región Occidente    4 cuentas  saldo  35303.00  cargos cobrados  20.00
      Sucursal Centro   2 cuentas  saldo   1320.00  cargos cobrados   0.00
      Sucursal Playa    2 cuentas  saldo  33983.00  cargos cobrados  20.00
    Sucursal Empresas   1 cuenta   saldo  33719.00  cargos cobrados  81.00

  CalcularIntereses sobre el banco entero (mismo visitante, sin cambios): total 183.89

  Cobro de mantenimiento de 5.00 a todas las cuentas del banco (mejor esfuerzo):
```

```
  Rechazada  Pago de Mantenimiento por 5.00 desde AHO-001
             motivo: AHO-001: alcanzó el límite de 3 retiros del mes
  Aplicada   Pago de Mantenimiento por 5.00 desde COR-001
  Aplicada   Pago de Mantenimiento por 5.00 desde INV-001
  Rechazada  Pago de Mantenimiento por 5.00 desde AHO-002
             motivo: AHO-002: saldo insuficiente (saldo 3.00, a debitar 5.00)
  Aplicada   Pago de Mantenimiento por 5.00 desde EMP-001
  → 3 aplicadas y 2 rechazadas; las rechazadas no tocaron sus cuentas.

--- 6. Iterator: historial de COR-001 ---
  Retiro de 600.00 de COR-001                    Debito  efecto  -605.00  (revertido)
  Transferencia de 50.00 de COR-001 a INV-001    Debito  efecto   -55.00  (revertido)
  Pago de Mantenimiento por 5.00 desde COR-001   Debito  efecto    -5.00

  Solo las transferencias de COR-001:
  Transferencia de 50.00 de COR-001 a INV-001
```

- **Los totales cuadran por niveles:** Centro (1320.00) más Playa (33983.00) dan la Región (35303.00), y la Región más Empresas (33719.00) dan el Banco (69022.00).
- **El cobro sigue la política del punto 2:** las 3 cuentas que pueden pagar pagan; AHO-001 (ya hizo 3 débitos este mes) y AHO-002 (sin saldo) quedan intactas y aparecen en el reporte con su motivo.
- **El historial de COR-001** muestra los dos débitos revertidos del inciso b y el cobro de mantenimiento; el filtro por tipo deja solo la transferencia.

2.3.5. Pruebas

`dotnet test` ejecuta **57 pruebas**, todas correctas: las 46 del inciso b y 11 nuevas.

Prueba	Qué comprueba
`Aceptar_RecorreLosGruposAnidados-EnProfundidadYEnOrden`	Un grupo anidado en tres niveles recorre todas sus hojas en profundidad y en orden, con los avisos de entrada y salida.
`Agregar_RechazaLosCiclos`	No se puede meter un grupo dentro de sí mismo ni dentro de un descendiente.
`Hijos_NoExponeLaListaInterna`	Los hijos se recorren sin poder modificar la lista.
`ResumirPorGrupo_SumaCadaCuentaEn-TodosLosGruposQueLaContienen`	Totales de saldo y cargos correctos en cada nivel de un árbol anidado.
`UnVisitanteQueNoDistingueGrupos_-FuncionaSobreUnGrupoSinCambios`	`CalcularIntereses` recorre un grupo entero sin ningún cambio.
`EjecutarEnCadaCuenta_ConFalloParcial-SigueConLasDemasYReportaCadaResultado`	**Fallo parcial:** la cuenta que rechaza queda intacta, las demás se cobran y el reporte da cada resultado.
`EjecutarEnCadaCuenta_TrataIgual-AUnaCuentaSola`	La misma operación funciona con una cuenta sola.
`Historial_RecorreLosAsientosEnOrden-IncluidosLosRevertidos`	El historial sale en orden, incluidos los asientos revertidos.
`Historial_NoExponeLaListaInterna`	El historial no se puede convertir en una colección modificable.
`Historial_FiltraPorFecha`	El filtro por fecha devuelve solo los asientos del intervalo.
`Historial_FiltraPorTipoDeTransaccion`	El filtro por tipo devuelve solo las transferencias.

Tabla 43: Pruebas nuevas del inciso c.

Referencias. Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. Los números de página corresponden a la copia del libro en la bibliografía de la asignatura (*Documentacion Patrones*), igual que en los incisos a y b.

Ingeniería de Software. *Marcos de Trabajo y Patrones de Diseño* [diapositivas de la conferencia].

2.4. d) Comparación

> *Visitor vs. añadir métodos a cada elemento. ¿Cuándo conviene cada uno? Usa la métrica de «número de cambios por nueva operación» y «número de cambios por nuevo tipo de elemento». ¿Qué pasa si el número de operaciones crece mucho más rápido que el número de tipos de cuenta?*

Las dos alternativas son:

- **Visitor**, la solución de los incisos a–c: cada operación es una clase visitante con un `Visitar` por tipo de cuenta.
- **Métodos en cada elemento**, el «polimorfismo simple» del inciso a: cada operación es un método abstracto en `Cuenta` que cada cuenta redefine.

Se comparan con las dos métricas del enunciado, contando clases nuevas y clases existentes modificadas. Las cifras salen del propio proyecto: E = 4 tipos de cuenta, y O = 3 operaciones del enunciado (más las que se agregaron en los incisos b y c).

2.4.1. Las dos métricas

	Cambios por nueva operación	**Cambios por nuevo tipo de cuenta**
Visitor	1 clase nueva; **0** clases modificadas	1 clase nueva + la interfaz del visitante + los **O** visitantes modificados
Métodos en cada elemento	0 clases nuevas; la clase base + las **E** cuentas modificadas	1 clase nueva que implementa las O operaciones; **0** clases modificadas

Tabla 44: Cambios por nueva operación y por nuevo tipo de cuenta.

Las dos métricas se oponen: lo que una alternativa hace fácil, la otra lo hace difícil. GoF lo resume así: Visitor facilita agregar operaciones (Gamma et al., 1994, p. 370) y dificulta agregar elementos, porque cada uno obliga a tocar todos los visitantes (p. 371).

El trabajo de escribir código es el mismo en las dos: cada combinación de operación y tipo de cuenta necesita su cuerpo de método (un `Visitar` o un `override`). Lo que cambia es **dónde** va ese código:

- en clases **nuevas**, que no rompen nada de lo que ya funciona y se prueban solas;
- o **modificando** clases que ya funcionan, con su riesgo de romperlas, de volver a probarlas y de chocar con otros cambios en las mismas clases.

Medido en este proyecto.

- **Nueva operación con Visitor.** `CalcularIntereses` (inciso b) está en un commit aparte, `94e3d6a`, y `git show --stat 94e3d6a` muestra 3 ficheros: la clase nueva, sus pruebas y su sección en `Program.cs`. **Ninguna cuenta cambió.** Lo mismo pasó con `ResumirPorGrupo` y `RecolectarCuentas` en el inciso c: una clase nueva cada una.
- **Nuevo tipo de cuenta con Visitor.** Se probó a agregar una `CuentaNomina` hipotética con su `Visitar` en la interfaz. **El compilador marcó 7 clases que había que cambiar:**

- los 6 visitantes (`EjecutarTransaccion`, `RevertirTransaccion`, `AuditarTransaccion`, `CalcularIntereses`, `ResumirPorGrupo` y `RecolectarCuentas`);
- el buscador de ciclos interno de `GrupoCuentas`.

Hay que sumarles la interfaz y los visitantes de prueba. Hoy, con O = 6, un tipo de cuenta nuevo cuesta 1 clase nueva y 8 modificadas. Que el programa no compile hasta completarlos es una ventaja: ninguna operación se queda sin regla para la cuenta nueva.

- **El mismo cambio con métodos en cada elemento.** Una operación nueva como `CalcularIntereses` habría modificado `Cuenta` y las 4 cuentas: 5 clases (6 si la operación también se aplica a los grupos del inciso c). La `CuentaNomina`, en cambio, sería 1 clase nueva y 0 modificadas.

Un caso intermedio: el Composite del inciso c. `GrupoCuentas` es un tipo de elemento nuevo, y con Visitor puro habría obligado a tocar todos los visitantes. No hizo falta, porque la interfaz ganó `EntrarEnGrupo` y `SalirDeGrupo` **con una implementación vacía por omisión** (C# lo permite desde la versión 8). El costo fue 1 clase nueva y 1 interfaz modificada; ningún visitante cambió.

GoF admite esta salida, pero como excepción (p. 371): funciona cuando existe un comportamiento razonable por omisión. Para un grupo lo hay: «no hacer nada al entrar ni al salir». Para una cuenta nueva no: cada operación tiene que decidir su regla, y ahí conviene que el compilador obligue a escribirla.

2.4.2. Escenarios de crecimiento

Partiendo del enunciado (E = 4, O = 3):

Escenario	Visitor	Métodos en cada elemento
Agregar 5 operaciones (congelar cuenta, calcular intereses, cierre de mes, estado de cuenta, cobrar mantenimiento)	5 clases nuevas, **0 modificadas**	0 nuevas; **25 modificaciones**: 5× (base + 4 cuentas), cada cuenta tocada 5 veces
Agregar 2 tipos de cuenta (nómina, plazo fijo)	2 clases nuevas; la interfaz y los 3 visitantes modificados (**4 clases**, cada visitante con 2 `Visitar` nuevos)	2 clases nuevas, **0 modificadas**

Tabla 45: Escenarios de crecimiento con E = 4 y O = 3.

2.4.3. ¿Cuándo conviene cada uno?

Visitor conviene cuando los tipos de elemento son estables y las operaciones cambian a menudo. Es el caso de este banco:

- un banco crea productos nuevos (tipos de cuenta) muy de vez en cuando;
- el enunciado anticipa operaciones nuevas («congelar cuenta», «calcular intereses»);
- en este mismo proyecto se agregaron 3 operaciones (intereses, resumen por grupo y recolectar cuentas) y ningún tipo de cuenta.

GoF lo pone como condición del patrón: que las clases de la estructura rara vez cambien (p. 368).

Métodos en cada elemento conviene en tres situaciones:

- **Cuando los tipos crecen y las operaciones son estables.** Por ejemplo, un catálogo de productos financieros que suma tipos de cuenta cada mes, siempre con las mismas tres operaciones.
- **Cuando las operaciones necesitan mucho estado interno de la cuenta.** Con Visitor, la cuenta tiene que exponer ese estado al visitante, lo que debilita la encapsulación (p. 372). Aquí se pagó ese precio con los contadores: lectura pública y `set` interno.
- **Cuando hay una o dos operaciones fijas.** En ese caso, la indirección del doble despacho no aporta nada.

2.4.4. ¿Qué pasa si las operaciones crecen mucho más rápido que los tipos de cuenta?

Visitor escala mucho mejor. Su costo por operación nueva es **constante**: 1 clase nueva y 0 modificadas, sin importar cuántas cuentas haya. Con métodos en cada elemento, el costo por operación **crece con E**, porque cada operación nueva toca la base y todas las cuentas.

Por ejemplo, en un año con 10 operaciones nuevas y 1 tipo de cuenta nuevo, empezando con E = 4 y O = 3:

	Visitor	**Métodos en cada elemento**
10 operaciones nuevas	10 clases nuevas, 0 modificadas	$10 \times 5 =$ **50 modificaciones**
1 tipo de cuenta nuevo	1 clase nueva + interfaz + entre 3 y 13 visitantes modificados (según cuándo llegue)	1 clase nueva, 0 modificadas
Total de modificaciones	**entre 4 y 14**	**50**

Tabla 46: Un año en el que las operaciones crecen mucho más rápido que los tipos de cuenta.

La otra cara: con Visitor, cada tipo de cuenta nuevo cuesta más cuanto más operaciones haya ($1 + O$ modificaciones). Si algún día los tipos de cuenta empezaran a crecer tan rápido como las operaciones, la ventaja se invertiría. GoF lo plantea como la pregunta clave para elegir el patrón: qué es más probable que cambie, los algoritmos o las clases de la estructura (p. 371).

Este dilema tiene nombre: el **problema de la expresión** (*expression problem*, Wadler, 1998). Consiste en extender a la vez los tipos de datos y las operaciones sin modificar el código existente. La programación orientada a objetos clásica facilita agregar tipos; Visitor invierte la balanza y facilita agregar operaciones. Las implementaciones por omisión de las interfaces, como en el inciso c, suavizan el costo cuando existe un comportamiento razonable por omisión.

Referencias. Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. Los números de página corresponden a la copia del libro en la bibliografía de la asignatura (*Documentacion Patrones*), igual que en los incisos anteriores.

Wadler, P. (1998). *The expression problem* [mensaje a la lista de correo Java-genericity].

2.5. Restricción

> *Las operaciones deben poder ejecutarse en paralelo sobre diferentes cuentas (considera concurrencia y estado mutable del Visitor). Además, el sistema debe garantizar que las transacciones sobre una misma cuenta sean atómicas y consistentes (¿cómo lo resuelves con el patrón elegido?).*

Se construye sobre el código de los incisos b y c. Todo el cambio está en `ProcesadorTransacciones` (el cliente del patrón) y en un candado por cuenta: **ningún visitante cambió**.

2.5.1. Respuesta corta

Pregunta	Respuesta
¿Cómo corren en paralelo las operaciones sobre cuentas distintas?	`EjecutarEnParalelo` reparte las transacciones con `Parallel.For`. Cada transacción solo bloquea sus propias cuentas, así que las de cuentas distintas no se esperan entre sí.
¿Qué pasa con el estado mutable del visitante?	`EjecutarTransaccion` y `RevertirTransaccion` no tienen estado mutable y se pueden compartir. `AuditarTransaccion`, `CalcularIntereses` y `ResumirPorGrupo` acumulan resultados: se usa **una instancia por tarea** y los resultados se combinan al final.
¿Cómo se garantiza que las transacciones sobre una misma cuenta sean atómicas y consistentes?	**Un candado por cuenta.** Lo toma el cliente del patrón, que envuelve las visitas, **en un orden fijo** (por número de cuenta) y no lo suelta hasta terminar, incluida la reversión si algo falla.
¿Cómo se resuelve con el patrón elegido?	Visitor separa las reglas de negocio (los visitantes) de quién recorre las cuentas (el cliente). La concurrencia es responsabilidad de ese cliente: los visitantes siguen sin saber nada de hilos.

Tabla 47: Respuesta corta a la restricción.

2.5.2. Paralelismo sobre cuentas distintas

```
public IReadOnlyList<ResultadoTransaccion> EjecutarEnParalelo(IReadOnlyList<Transaccion>
    transacciones)
{
    ResultadoTransaccion[] resultados = new ResultadoTransaccion[transacciones.Count];
    Parallel.For(0, transacciones.Count, i => resultados[i] = Ejecutar(transacciones[i]));
    return resultados;
}
```

- **Cada `Ejecutar` toma solo los candados de sus cuentas.** Dos transacciones sobre cuentas distintas no comparten ningún candado y corren a la vez. Dos que comparten una cuenta se ejecutan de una en una.
- **Los resultados vuelven en el orden de la lista,** aunque los hilos terminen en cualquier orden: cada tarea escribe en su posición del arreglo.
- **Las operaciones sobre un grupo (inciso c) también corren en paralelo.** `EjecutarEnCadaCuenta` crea una transacción por cuenta del grupo y las ejecuta con `EjecutarEnParalelo`. Como cada una toca una sola cuenta, ninguna espera a otra.

2.5.3. El estado mutable de los visitantes

GoF describe al visitante concreto como el que guarda el estado local del algoritmo, que a menudo acumula resultados durante el recorrido (Gamma et al., 1994, p. 369). Ese estado es el que no se puede compartir entre hilos:

Visitante	Estado mutable	¿Compartible?	Solución
`EjecutarTransaccion`	ninguno: solo guarda la transacción, que no cambia	sí	se crea uno por transacción de todos modos
`RevertirTransaccion`	ninguno	sí	ídem
`AuditarTransaccion`	la lista de registros	**no**	una instancia por tarea (`AuditarEnParalelo`)
`CalcularIntereses`	el diccionario de intereses	**no**	una instancia por recorrido
`ResumirPorGrupo`	los acumuladores por grupo	**no**	una instancia por recorrido
`RecolectarCuentas`	la lista de cuentas	**no**	`RecolectarCuentas.De` crea uno en cada llamada

Tabla 48: Estado mutable de cada visitante y cómo se evita la condición de carrera.

Qué pasaría si se compartiera un `AuditarTransaccion` entre hilos: dos hilos llamando a `List.Add` a la vez pueden perder registros, sobrescribirse o lanzar una excepción. Lo mismo pasa con `+=` en un acumulador: leer, sumar y escribir son tres pasos, y dos hilos pueden leer el mismo valor viejo.

Solución elegida: una instancia por tarea, y combinar al final.

```
public IReadOnlyList<RegistroAuditoria> AuditarEnParalelo(IReadOnlyList<Transaccion>
    transacciones)
{
    IReadOnlyList<RegistroAuditoria>[] porTarea = new IReadOnlyList<RegistroAuditoria>[
    transacciones.Count];
    Parallel.For(0, transacciones.Count, i => porTarea[i] = Auditar(transacciones[i]));  // cada
     Auditar crea su visitante
    return porTarea.SelectMany(registros => registros).ToList();                        // se
    combinan al final, en orden
}
```

Se descartó usar colecciones concurrentes (`ConcurrentBag`) dentro del visitante por dos razones:

- **Perderían el orden de los registros,** y la auditoría lo necesita.
- **No resolverían lo importante.** El estado compartido que de verdad importa no está en los visitantes, sino en las **cuentas** (saldo, contadores e historial), y eso lo protege el candado del punto siguiente.

2.5.4. Atomicidad y consistencia en la misma cuenta

Un candado por cuenta. Cada `Cuenta` tiene un objeto privado, `Candado`, y `ProcesadorTransacciones` lo toma con `Monitor` (lo que usa `lock` en C#) antes de pasar el visitante.

- **Es un objeto propio y no la cuenta misma,** para que ningún código de fuera pueda bloquearlo por accidente.
- **Todo lo que cambia la cuenta pasa con el candado tomado:** aplicar el movimiento, cobrar el cargo, sumar el contador y anotar el asiento. Ningún otro hilo ve la cuenta a medio cambiar.

Dónde se toma: en el cliente que envuelve las visitas, no en `Aceptar`. Esta es la respuesta a «¿cómo lo resuelves con el patrón elegido?». Visitor ya separa dos responsabilidades: los visitantes tienen las reglas de cada tipo de cuenta, y el cliente decide qué cuentas visitar y con qué visitante. La atomicidad es una propiedad de la **transacción entera**, que puede tocar varias cuentas, así que el candado lo toma quien conoce todas esas cuentas: el cliente.

```
public ResultadoTransaccion Ejecutar(Transaccion transaccion) =>
    ConCandados(transaccion, () => EjecutarConCandados(transaccion));   // candados alrededor de
      todas las visitas
```

Tomar el candado dentro de `Aceptar` de cada cuenta no alcanzaría, por tres razones:

1. **Una transferencia necesita las dos cuentas a la vez.** Con el candado en `Aceptar`, la cuenta de origen se soltaría después del débito y antes del crédito, y otro hilo podría ver el débito hecho y el crédito no. Si el crédito se rechaza, la reversión del débito correría sin protección.

2. **No se podría fijar el orden de los candados.** Cada `Aceptar` tomaría el suyo en el orden de los movimientos, que es justo lo que provoca interbloqueos (punto siguiente).

3. **Mezclaría responsabilidades.** Las cuentas (elementos) y los visitantes (reglas) no tienen por qué saber de hilos. Con el candado en el cliente, `EjecutarTransaccion` sigue suponiendo que tiene la cuenta para él solo, y eso es cierto.

Orden fijo: sin interbloqueo.

```
Cuenta[] cuentas = [.. transaccion.Cuentas.OrderBy(c => c.Numero, StringComparer.Ordinal)];
// se toman en ese orden, se ejecuta la acción y se sueltan en orden inverso
```

Qué pasaría sin el orden fijo: dos transferencias cruzadas, A→B y B→A, al mismo tiempo. La primera toma A y espera B; la segunda toma B y espera A. Ninguna avanza nunca.

Con el orden fijo: las dos piden primero el candado de la cuenta de número menor. La que lo consigue toma también el otro, termina y suelta los dos; la otra espera y después sigue. Así se elimina la espera circular, la condición que hace falta para que haya interbloqueo. Por eso el número de cuenta es único.

Si falla a mitad: todo o nada con `RevertirTransaccion`. Si el débito ya se aplicó y el crédito se rechaza, el procesador deshace el débito con el mismo `RevertirTransaccion` del inciso b, **todavía con los candados tomados**. Nadie llega a ver el estado intermedio: desde fuera, la transacción se aplicó entera o no se aplicó.

Por indicación de la revisión del inciso b, si deshacer también fallara, el procesador ya no tapa el error original: lanza una `AggregateException` con los dos errores.

Lecturas consistentes.

- **`Auditar` y `Revertir` también toman los candados de sus cuentas,** así que no leen ni deshacen una cuenta a medio cambiar.

- **El historial no se puede recorrer mientras otra transacción escribe en la cuenta.** El iterador del inciso c falla si la lista cambia durante el recorrido, en vez de devolver datos a medias. Para leerlo con transacciones en curso hay que hacerlo con el candado de la cuenta tomado, como hace la auditoría.

2.5.5. Límites y alternativas

- **`Monitor` es síncrono.** Los visitantes no esperan nada (solo hacen cuentas), así que alcanza. Si una regla necesitara `await`, por ejemplo para consultar un servicio externo, haría falta `SemaphoreSlim(1, 1)` con `WaitAsync`, porque un `Monitor` no se puede mantener a través de un `await`.
- **El candado es por cuenta, no global.** Una cuenta muy activa hace esperar solo a sus propias transacciones. Un candado global sería correcto, pero quitaría todo el paralelismo.

2.5.6. Demostración

La sección 7 de la simulación lanza 1000 transacciones a la vez en cada caso. La salida no depende del orden en que corran los hilos:

```
--- 7. Restricción: transacciones en paralelo ---
  1000 depósitos y retiros de 10.00 a la vez sobre COR-101: 1000 aplicados, saldo 10000.00 (esperado
     10000.00), 1000 asientos
  1000 transferencias cruzadas entre COR-101 y COR-102 a la vez: 1000 aplicadas, sin interbloqueo; suma de
     las dos cuentas 20000.00 (se conserva)
  Auditoría en paralelo de esas transferencias, un AuditarTransaccion por tarea: 2000 registros (2 por
     transferencia), 2000 conformes
```

`dotnet test` ejecuta **62 pruebas**, todas correctas: las 57 de los incisos b y c y 5 de concurrencia. Las ejecuté 5 veces seguidas sin ningún fallo.

Prueba	Qué comprueba
`DepositosYRetirosEnParalelo-SobreLaMismaCuenta_DanElSaldoEsperado`	1000 depósitos y retiros a la vez sobre **la misma** cuenta: el saldo final es el esperado y no se pierde ningún asiento.
`TransferenciasCruzadasEnParalelo_-TerminanSinInterbloqueoYConservanElDinero`	1000 transferencias A→B y B→A a la vez terminan antes de 30 s (sin interbloqueo) y la suma de las dos cuentas no cambia.
`TransferenciasRechazadasEnParalelo_-NoDejanRastroEnLaCuentaDeOrigen`	500 transferencias que fallan a mitad, mezcladas con 500 depósitos: el origen solo refleja los depósitos (todo o nada bajo concurrencia).
`ReversionesEnParaleloSobreLaMismaCuenta_-DejanElEstadoOriginal`	1000 retiros y sus 1000 reversiones en paralelo devuelven el saldo, las comisiones y las autorizaciones originales.
`AuditoriasEnParalelo_ConUnVisitante-PorTarea_NoPierdenRegistros`	1000 auditorías en paralelo dan los 1000 registros, en orden.

Tabla 49: Pruebas de concurrencia.

Referencias. Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley. Los números de página corresponden a la copia del libro en la bibliografía de la asignatura (*Documentacion Patrones*), igual que en los incisos anteriores.